QCert: Verifying and Extending Quantum Circuit Completeness in Agda and Lean

ANONYMOUS AUTHOR(S)

A set of circuit equations is complete if every equation between circuits denoting the same linear map follows from it, and minimal if no equation can be derived from the others. Establishing these properties takes long case analyses spanning hundreds of pages, making the details hard to check by hand. To address this, we extend an existing formalization library to QCert in two proof assistants. In QCert, humans set up a unified framework of circuits, matrix semantics, and group presentations, and state each paper's main theorem using these definitions. LLM coding agents then write the proofs following the papers' arguments, and the proof assistants check them. A human therefore need only review each paper's relations, gate semantics, and theorem statement. Using QCert, we first check sixteen published results, fifteen on completeness and one on minimality. They cover discrete gate sets such as Clifford, Clifford+T, and Toffoli–Hadamard, on qubits or qudits of every odd prime dimension. We verify fourteen of them directly and correct an error in each of the other two. We also obtain a complete equational theory for circuits over $\{X, CX, CCX\}$, and extend the completeness of Toffoli–Hadamard circuits from three qubits to any number of qubits. We expect QCert to help with other open completeness problems, such as those for quantum circuits in higher dimensions.

1 Introduction

Rewriting of quantum circuits is a ubiquitous tool in circuit optimization [41, 50, 51, 65], equivalence checking [66], hardware mapping [63], and reasoning about quantum programs [31]. This relies on an *equational theory* for a family of quantum circuits, a set of rewrite rules that transform a circuit into an equivalent one. Such a theory should be *sound* so that every rewrite preserves the underlying linear map. It should also be *complete* so that any equivalence between circuits can be derived. Additionally, we want it to be *minimal*, with no redundant rules. Completeness and minimality proofs often involve long case analyses, with supporting derivations spanning hundreds of pages [24, 31, 44]. When a derivation takes many rewrite steps, one step can easily rely, through earlier lemmas, on the very equation being derived. A *proof assistant* can check these dependencies and reject such circular arguments.

Our Contributions. We present QCert, a framework in Agda [19, 53] and Lean [29] for representing and verifying quantum circuit completeness and minimality results. Figure 1 summarises its workflow: humans check if the formal definitions, relations, and theorem statement faithfully express the source paper; LLM coding agents formalize the proof, and Lean or Agda checks it.

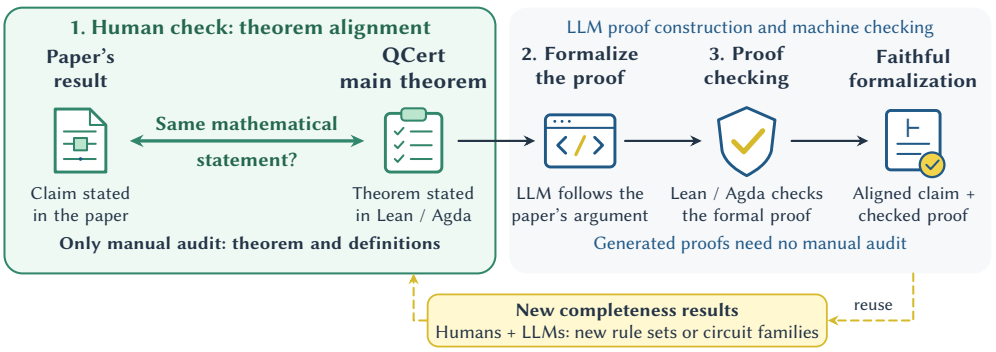


Fig. 1. QCert's workflow. The main path checks published results; the dashed branch reuses the verified libraries for human-guided, LLM-assisted work on new completeness results. Proposed results undergo the same human review of definitions and theorem statements, and the same machine checking of formal proofs.

The manual audit matters because a proof assistant checks a proof only against the theorem as stated, which may not match the intended result [40]. For example, Bastounis et al. [8] find that, because of mistranslations, a claimed Navier–Stokes formalization verified a weaker statement than intended. QCert therefore states each paper's main theorem in its own module, in both Agda and Lean. Since the proof assistant checks every proof, a reviewer need only compare these statements, and the definitions they use, with the source papers to confirm that a formalization is faithful.

QCert builds on two earlier Agda developments. The first is the formalization [11] of the multi-qudit Clifford completeness result of Bian, Li et al. [12]. The second is the library of circuits and group presentations of Bian and Feng [10], which QCert generalizes by adding matrix semantics, minimality, three-wire gates, and scalar generators. Once the Agda framework was complete, we instructed LLM coding agents to translate it into Lean, which has a large community that uses it to formalize mathematics [54, 64], build AI theorem provers [67], and formalize quantum information theory and quantum programs [9, 42, 62, 70]. As a result, QCert can build on Lean's well-developed mathematical library Mathlib. In the long run, the Lean community can use QCert's verified rule sets and completeness theorems to justify circuit rewrites in a verified quantum program.

We then use QCert to check sixteen papers, which include most of the completeness results for quantum circuits over discrete gate sets. We verify fourteen of them and correct errors that peer review had missed in the other two. Most of these circuit families are restricted Clifford+T circuits. Amy et al. [2] show that several of these circuit families, with at most one ancilla, represent exactly the unitary matrices over a subring of $\mathbb{Z}[1/\sqrt{2}, i]$. Figure 2 displays these gate sets, their matrix groups, and multi-level generators. Because these families are nested and their matrices all have entries in $\mathbb{Z}[1/\sqrt{2}, i]$, QCert reuses exact arithmetic, matrix semantics, group constructions, and completeness arguments across formalizations.

Beyond verifying published results, QCert facilitates the discovery of new ones by freeing researchers from labour-intensive checking so they can focus on developing new techniques. Until now, Toffoli–Hadamard circuits had a complete equational theory only on three qubits [3], and its rule set omits a relation that QCert shows to be independent. Working with LLM coding agents, we combine the presentation of $O_n(\mathbb{Z}[1/2])$ by Li et al. [45] with the method of Clément [24] to extend this completeness to any number of qubits. Along the way, we obtain a complete equational theory for circuits over $\{X, CX, CCX\}$. QCert then checks and confirms both proofs, building on its existing formalizations of the presentation and the method they combine.

Background and Related Work. Completeness results have been established for a wide range of circuit families, including reversible linear circuits [43, 58], reversible circuits over multiple-control Toffoli gates [32, 33], Clifford [12, 44, 49, 60], CNOT-dihedral [1, 17], two-qubit Clifford+T [14], three-qubit Clifford+CS [15], Toffoli–Hadamard [3], and real Clifford+CH [24] circuits, as well as universal quantum circuits [26, 27]. Completeness has also been obtained for the group of two-level matrices underlying several of these circuit families [13, 23, 31, 36, 45]. Minimal theories have been given for six of these circuit fragments [18] and for all quantum circuits [25]. Their proofs typically follow a simple strategy, such as rewriting every circuit into a unique normal form, but carrying it out requires overwhelming case distinctions [24, 31, 44].

Peer review cannot check proofs of this volume in detail, so it is desirable to verify them in a proof assistant such as Lean or Agda. This would not have been feasible in the past due to the sheer number of lemmas and lengthy rewrites in these papers. Recent LLMs can write correct code for proof assistants, which brings this closer to reality. For example, Anthropic reports that LLMs wrote a 13-million-line Lean proof of Fermat's Last Theorem within 11 days [7].

In quantum computing, proof assistants and, more recently, LLM agents have mostly been applied to programs rather than to circuit equational theories. Mechanized quantum compilers and program

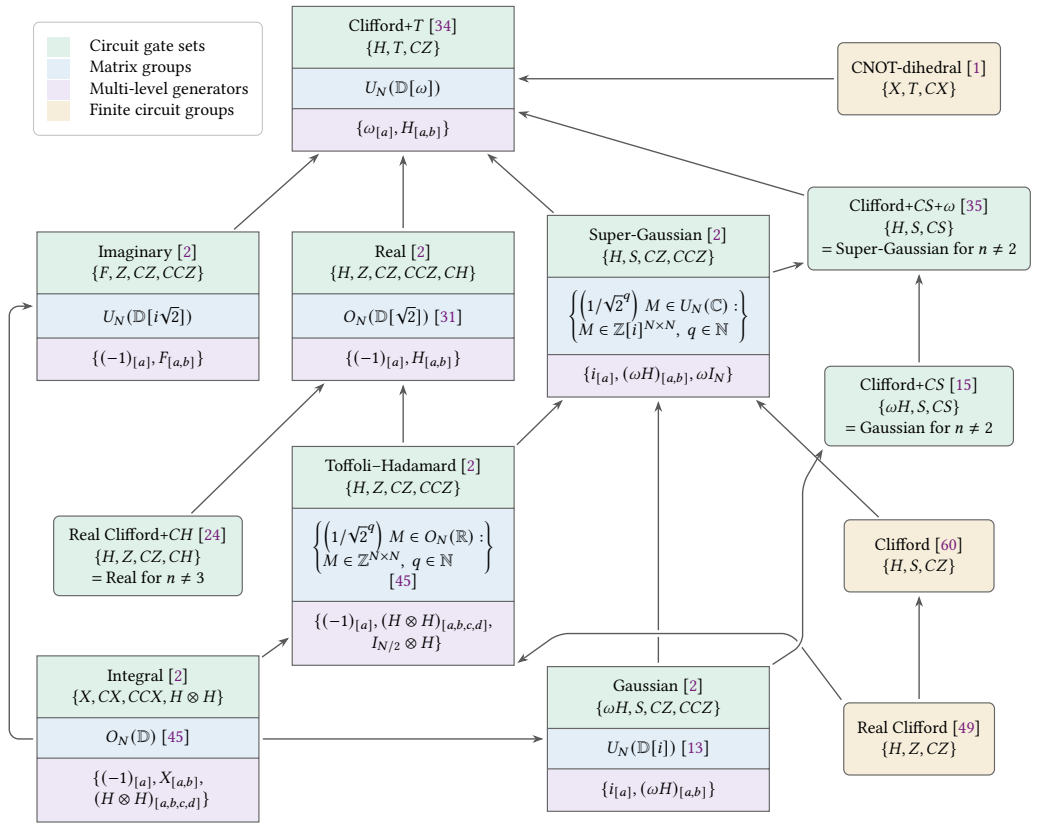


Fig. 2. Restricted Clifford+T circuit families and their underlying matrix groups, with $N = 2^n$ and $n \geq 1$. Each matrix group is the one its circuits represent with at most one clean ancilla. Arrows denote inclusion of ancilla-free circuit groups and, where shown, matrix groups. Mint-only boxes mark when related circuit groups agree; each exceptional inclusion is strict. Here $\mathbb{D} = \mathbb{Z}[1/2]$, $\omega = e^{i\pi/4}$, $F = (H + iI)/\sqrt{2}$, and q ranges over nonnegative integers. Subscripts specify distinct basis levels; $I_{N/2} \otimes H$ and ωI_N act globally. In both Gaussian cases, $X_{[a,b]} = (\omega H)_{[a,b]} i^2_{[b]} (\omega H)^\dagger_{[a,b]}$ is redundant. Clifford+CS+ω denotes adjoining the scalar ω to Clifford+CS.

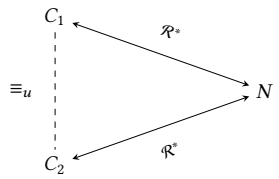
logics establish program correctness or the soundness of circuit transformations [21, 41, 55, 69]. The mechanized completeness results for Π and its square-root extension are coherence arguments [20, 22]. Human-supervised, AI-assisted workflows have also formalized quantum program logics, algorithms, and components of interactive proofs [46, 57, 68]. In concurrent work, Shirobokov et al. [62] build a quantum program library in Lean, where researchers write frozen specifications of correctness and cost and teams of LLM agents synthesize and verify new programs against them.

Orientation. Sections 2 and 3 introduce the proof strategies and the QCert framework. Section 4 presents the formalized results and compares the Agda and Lean formalizations, and Section 5 describes the corrections to the source papers. Section 6 presents a new completeness theorem for four-bit reversible circuits and its formalization. Section 7 discusses limitations and future work. Appendix D works through the multi-qudit Clifford formalization as a case study, and Appendix E proves the theorems of Section 6.

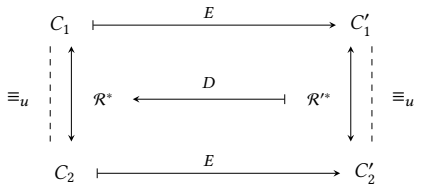
2 Preliminaries

A circuit is a word over an alphabet $\mathcal{G}$ of gates. A rewrite rule is then a relation between words over this alphabet, and a rule set $\mathcal{R}$ generates a congruence on circuits. To a circuit C we associate an *interpretation* $[\![C]\!]$, which is its underlying matrix. The completeness results we formalize have matrix entries in the finitely generated rings $\mathbb{Z}\left[1/\sqrt{2}, i\right]$, $\mathbb{Z}\left[1/2, i\right]$, $\mathbb{Z}\left[1/\sqrt{2}\right]$, and $\mathbb{Z}\left[1/2\right]$, which make them exactly representable in Agda. We leave the formalization of universal circuits taking values in $\mathbb{C}$ for future work.

There are two main strategies for proving completeness of a rule set $\mathcal{R}$ over a gate set $\mathcal{G}$, both of which we support. Consider any two circuits C_1 and C_2 over $\mathcal{G}$ that have the same linear map, $[\![C_1]\!] = [\![C_2]\!]$. We need to show that C_1 can be rewritten into C_2 using $\mathcal{R}$. Figure 3a outlines the first strategy. It specifies a unique normal form for each linear map [1, 12, 17, 32, 33, 43, 44, 49, 58, 60], often by giving an exact synthesis algorithm [13, 23, 31, 36, 45]. Completeness follows by proving that every circuit can be transformed into its normal form using the rewrite rules. By uniqueness, C_1 and C_2 have the same normal form N. Reversing the path from C_2 to N and concatenating it with the path from C_1 to N then gives the desired path of rewrites.



(a) The rewrites from C_1 and C_2 to their common normal form N using $\mathcal{R}$ give rewrites from C_1 to C_2 using $\mathcal{R}$.



(b) Completeness of $\mathcal{R}'$ gives rewrites between $C_1' = E(C_1)$ and $C_2' = E(C_2)$. Translating this proof back gives rewrites between C_1 and C_2 using $\mathcal{R}$.

Fig. 3. Two strategies for proving completeness of $\mathcal{R}$ over a gate set $\mathcal{G}$. $\overset{\mathcal{R}^*}{\longleftrightarrow}$ means applying a finite sequence of rewrites using $\mathcal{R}$ in either direction, where $*$ denotes zero or more steps.

Figure 3b illustrates the second strategy, which uses an already complete rule set $\mathcal{R}'$ over a different gate set $\mathcal{G}'$ [3, 14–16, 18, 24, 26, 27, 30, 59]. Here, E and D translate circuits from $\mathcal{G}$ to $\mathcal{G}'$ and back, replacing each gate with a circuit over the other gate set. Both E and D preserve circuit composition and the underlying linear map. To transfer completeness from $\mathcal{R}'$ to $\mathcal{R}$, we first show that every rule of $\mathcal{R}'$, translated by D, can be derived using $\mathcal{R}$. We can then replace each step of the proof between C_1' and C_2' by its translated derivation, giving $D(C_1') \overset{\mathcal{R}^*}{\longleftrightarrow} D(C_2')$. It then remains to prove that the rewrite $C_j \overset{\mathcal{R}^*}{\longleftrightarrow} D(C_j')$ is derivable, for which it suffices to prove the round-trip rewrites $g \overset{\mathcal{R}^*}{\longleftrightarrow} D(E(g))$ for each $g \in \mathcal{G}$. Sometimes the two strategies are combined, as in the scaled dyadic extension of [45].

The exact synthesis results used by the first strategy present groups of matrices rather than circuits (the first block of Table 1). Their generators are *multi-level* matrices. For an $m \times m$ matrix M and distinct indices $a_1, \ldots, a_m$ in $\{0, \ldots, n-1\}$, the *m-level* matrix $M_{[a_1,\ldots,a_m]}$ is the $n \times n$ matrix that acts as M on the standard basis vectors $e_{a_1}, \ldots, e_{a_m}$ and as the identity on the others. For example, $(-1)_{[a]}$ negates e_a, and $\mathrm{X}_{[a,b]}$ exchanges e_a and e_b. Circuits on k qubits have matrices of dimension $n = 2^k$, but these groups are defined in every dimension n. We introduce multi-level matrices for two reasons. The gap we find in Section 5 is in a presentation by one- and two-level matrices [31], and our Toffoli–Hadamard completeness result builds on a presentation by one-, two-, and four-level matrices [45].

For proofs based on exact synthesis, QCert supplies the common induction argument. Each development proves a Main Lemma connecting the algorithm's reduction steps to the rewrite rules. This isolates the algorithm-specific case analysis—the main work in each formalization—from the reusable completeness proof. Appendix A gives the argument.

Starting from a complete rule set, we can remove a redundant rule by deriving it from the other rules, thereby preserving completeness. To show that the resulting rule set is minimal, we need to give, for each remaining rule R, an interpretation of circuits that every other rule preserves but R does not. Any derivation using the other rules preserves this interpretation, so R cannot be derived from them [18, 25].

3 The QCert Framework

Our Agda framework QCert states every completeness theorem as a group presentation. Gates are indexed by their arity, the number of wires on which they act. Each source relation is stated on its original wires, and shared *structural rules* allow it to be used inside a larger circuit. These rules are independent of the gate set and make explicit the circuit structure that source papers often leave implicit.

We associate to circuits a matrix semantics $[\![-]\!]$ with entries in a finitely generated ring such as $\mathbb{Z}\left[1/\sqrt{2}, i\right]$. This defines a group homomorphism to a unitary matrix group G, whose group structure is checked once per ring using Agda's standard library. QCert extends the interpretation from gates to circuits and proves the structural rules sound once for all gate sets. For soundness, each development then needs to check only its own rules.

Soundness and completeness say the relations present the image of $[\![-]\!]$, which is the subgroup of G generated by the gates. In Agda, mathematical structures such as groups are defined as records. The record $\mathcal{R}$ IsSubPresentationOf G contains the semantics $[\![-]\!]$, with proofs of soundness and completeness. The record $\mathcal{R}$ IsPresentationOf G adds surjectivity, so the relations present all of G.

Figure 4 summarizes the two obligations: checking each rule's matrix equality for soundness, and deriving a known complete rule set $\mathcal{T}$ from $\mathcal{R}$ for completeness.

Each formalization ends with one of these records. For example, Statement (1) states that the real stabilizer relations [49] present the subgroup generated by their gates, while Statement (2) states that the Hadamard-Π relations [31] present the specified orthogonal group. The fields in Statements (3) and (4) state the two directions: equal matrices imply a derivable circuit equation, and every derivable equation preserves the matrix.

```
presentation : ∀ n → (n CRel,_===_) IsSubPresentationOf U (2 ^ n)          (1)
presentation : ∀ n → (n MRel,_===_) IsPresentationOf (O n)                 (2)
complete : ∀ {n} {w v : Circuit n} → [[ w ]] ≈ᵁ [[ v ]] → ⊢ w ≈ v          (3)
sound : ∀ {n} {w v : Circuit n} → ⊢ w ≈ v → [[ w ]] ≈ᵁ [[ v ]]             (4)
```

The minimality statement in Statement (5) says that the rule set is complete on n wires and becomes incomplete there if any rule is removed [18].

```
minimal-complete : ∀ {n} → (n ⊢_===_) IsMinimal × (n ⊢_===_) IsComplete     (5)
```

For each presentation, a reader checks four components against the source paper: the transcribed relations, the target group, the matrix of each gate, and the assignment of $[\![-]\!]$ to the record. Each Agda development collects these components and its main theorem in a `MainTheorem` module. The file `MainTheorems.agda` restates the results; the corresponding Lean statements are in each development's `Theorem` module, which `QCert/Table1.lean` imports.

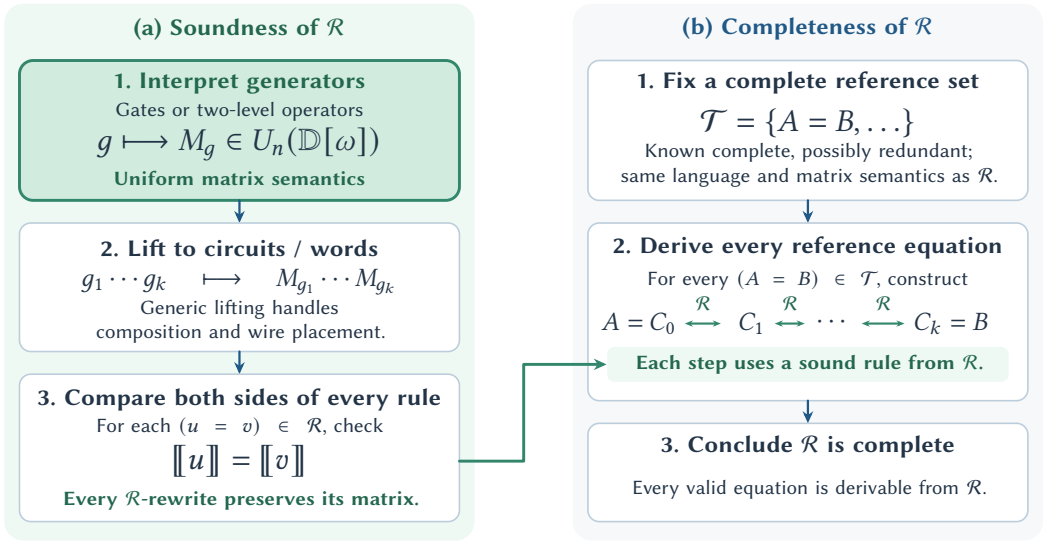


Fig. 4. QCert's shared matrix interpretation supports both proof obligations. The left column checks every rule of $\mathcal{R}$; the right derives every rule of a known complete set $\mathcal{T}$ in the same circuit language and matrix semantics. Each rewrite step uses a rule of $\mathcal{R}$, whose soundness is checked on the left. The displayed $U_n(\mathbb{D}[\omega])$ covers qubit fragments [2], with $\mathbb{D} = \mathbb{Z}[1/2]$ and $\omega = e^{i\pi/4}$. Qudit circuit families use other rings.

Extensions of the Earlier Framework. Besides matrix semantics and minimality, QCert extends the one- and two-wire syntax of Bian and Feng [10] with three-wire gates such as Toffoli and explicit scalar generators for expressing exact equality, including global phase. It also supplies the structural swap laws of PROPs [47], so each paper need only state its own relations. Appendix B gives the encoding conventions, theorem notation, and implementation details.

3.1 Formalization Workflow

Our workflow has three parts. First, before formalizing a paper, we check its rewrite rules, its intermediate results, and each line of its derivations with exact matrices in short scripts. This caught most typos before any Agda code was written. Second, we state each main theorem in QCert before a coding agent starts, so the agent only supplies the proof. Finally, for long derivations, the agent writes a script that turns each line of the paper into one step of the Agda proof, so a failing step points to a specific line of the paper.

Direct evaluation of long matrix products was expensive, so QCert uses a shared exact procedure for matrix equality. We record this technique in the agents' instructions so that later formalizations reuse it; Appendix B.3 gives the implementation.

3.2 Avoiding Completeness Lost in Translation

For a formalization to be useful, its formal statement must say what the source paper claims. We make this connection explicit in three ways.

First, we define the target group independently of the circuit syntax, using matrices or an algebraic construction. A completeness theorem then relates the circuit equations to a semantics that does not depend on them, rather than to a group built from the equations themselves.

Second, when a source rule set must be changed, the formal theorem names the revised set. This lets a reader tell a repair to the proof apart from a correction to the theorem itself (Section 5).

Finally, we verify general completeness arguments and group constructions once, including Reidemeister–Schreier coset enumeration [48] and presentations of semidirect products. Each formalization that uses them only needs to prove their hypotheses and check the connection to its own circuit semantics.

For example, Figure 5 shows how we formalize the multi-qudit Clifford result of Bian, Li et al. [12] using the first and third choices. Its semantic model is the semidirect product of the Pauli phase space by the symplectic group, which we define independently of the circuits. To prove completeness, we combine a normal-form proof for the symplectic group with the framework's semidirect-product and scalar-extension theorems. We give the full case study in Section D.

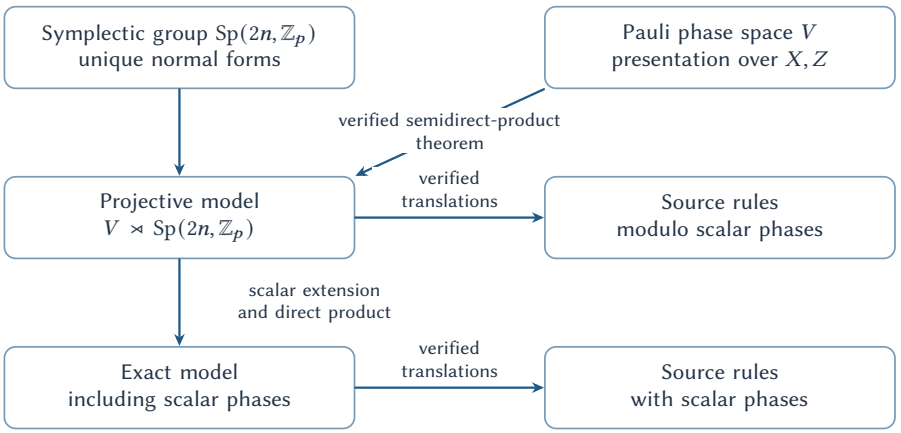


Fig. 5. The formalization of the multi-qudit Clifford result of Bian, Li et al. [12], assembled from verified components. The symplectic normal-form proof and the Pauli presentation combine into a projective model; restoring phases gives an exact model. Arrows denote checked constructions and translations of presentations. Appendix D gives the full case study, including the intermediate rule sets (Appendix D.6) and the scalar corrections (Appendix D.7).

4 Formalized Completeness and Minimality Proofs

The sixteen formalizations in Table 1 contain about 377,000 lines of Agda code and use no postulates, and each theorem is as general as the result in its source paper. Moreover, all matrices are computed exactly. Their entries lie in algebraic number rings, whose elements are given by finitely many integers, so Agda checks automatically whether two matrices are equal. Over the real or complex numbers, no such automatic check exists, which limits the equality checking of QCert to discrete gate sets, as we discuss in Section 7.

Figure 6 adds completeness status to the families and inclusions of Figure 2, using the survey's notation [56]. It separates circuit relations from presentations of the underlying matrix groups. QCert covers the integral, scaled-real, Gaussian, and real matrix presentations, but circuit completeness for Clifford+T and Clifford+CS is still restricted to small widths. A presentation of the ambient matrix group does not by itself provide relations for every circuit subgroup.

The green checks in Figure 6 correspond to the qubit part of Table 1; the qudit results and minimality proofs are also listed in the table. The asterisks connect this coverage to the source corrections in Section 5.

Source	Object	Result	Agda Mod	Agda time (s)	Lean time (s)
Groups of m-level matrices					
Greylyn [2014]	$U_n(\mathbb{Z}[1/\sqrt{2}, i])$	presentation, $n \leq 4$	34	37.28	145.34
Bian and Selinger [2021]	$U_n(\mathbb{Z}[1/2, i])$	presentation	39	53.73	126.99
Li et al. [2021]	$O_n(\mathbb{Z}[1/2])$, L_n	presentations	71	84.52	185.32
Fang et al. [2026]	$O_n(\mathbb{Z}[1/\sqrt{2}])$	presentation	46	48.33	231.57
Clément [2019]	$O_n(\mathbb{Z}[1/\sqrt{2}])$	presentation	43	67.72	51.99
Circuits					
Selinger [2015]	Clifford	sub-presentation	32	35.01	88.24
Bian, Li et al. [2026b]	Clifford, odd prime p	presentation of a model	216	499.37	197.32
Makary et al. [2021]	real stabilizer	sub-presentation	31	37.94	79.75
Amy et al. [2018]	CNOT-dihedral	sub-presentation	31	21.24	101.22
Blake [2026b]	phase-affine, prime d	sub-presentations	57	83.30	257.24
Bian and Selinger [2023a]	two-qubit Clifford+T	sub-presentation in U_4	63	135.79	356.79
Bian and Selinger [2023b]	three-qubit Clifford+CS	sub-presentation in U_8	125	298.29	1330.89
Clément [2026]	real Clifford+CH	sub-presentation	348	419.99	5478.79*
Amy et al. [2024]	three-qubit Toffoli–Hadamard	presentation of L_8	203	111.17	134.91
Li et al. [2025]	qutrit Clifford	presentation	90	285.58	179.60
Minimality					
Blake [2026c]	six fragments	minimal, complete	80	180.10	358.87
Total			1,509	2,399.36	9,304.83

Table 1. Completeness and minimality results formalized in QCert. A presentation gives generators and relations for a whole group, and a sub-presentation for the subgroup generated by the gate set named in the Object column of the Circuits block. L_n is the group of orthogonal matrices $N/\sqrt{2}^j$, where N is an integer matrix and $j \geq 0$. Mod counts Agda modules. The Agda times are typechecking times from scratch with Agda 2.8, the times its profiler reports for the row's modules; the Lean times are elapsed folder-build times. Both were measured on the same machine, under the conditions in Appendix C. Both columns exclude the shared library and other developments on which a row depends. The Result column describes the Agda statements; Appendix C explains the semantic differences. *Includes two unused legacy helpers that failed; the main theorem chain passed.

Two qualifications apply to the Agda results in Table 1. The two-qubit Clifford+T and three-qubit Clifford+CS developments [14, 15] reuse the authors' Agda proofs and connect them to QCert's shared matrix presentations (Section 3). The qudit Clifford theorem of Bian, Li et al. [12] presents an independently constructed algebraic model; Appendix D.3 explains its connection to the intended unitary semantics.

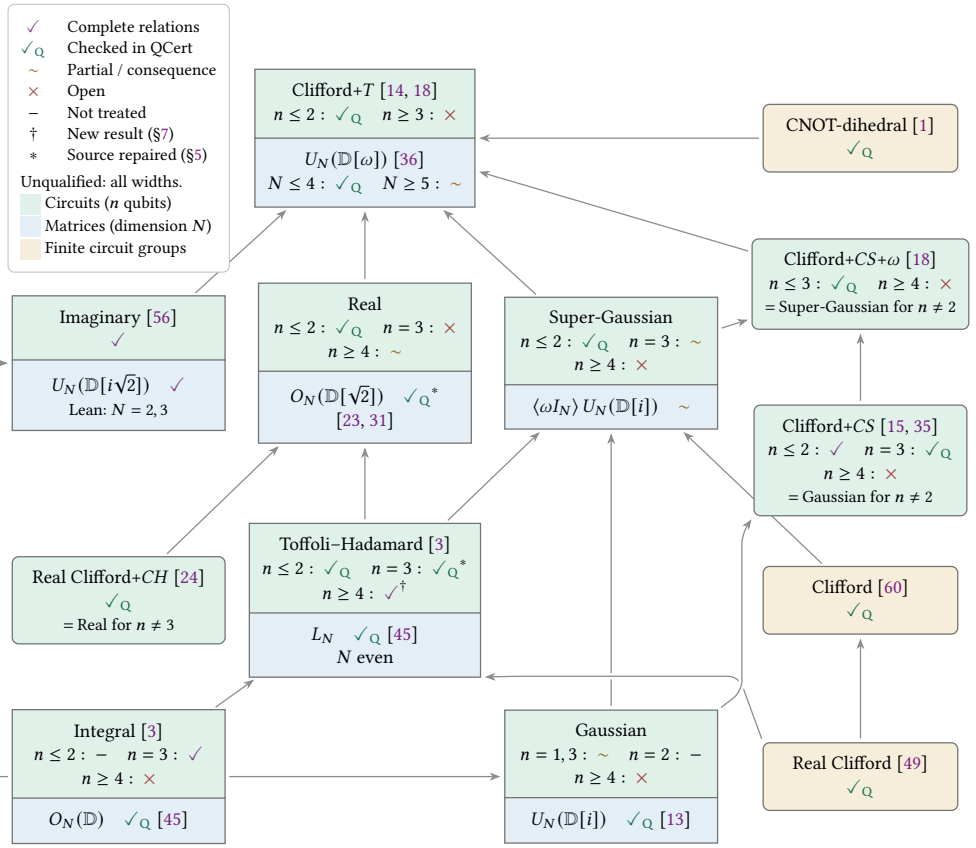


Fig. 6. Completeness progress on the same families and inclusion arrows as Figure 2. The dagger marks the new Toffoli–Hadamard result, whose exposition is in preparation. Gates and multi-level generators are omitted; upper compartments concern ancilla-free circuits, lower compartments matrix presentations. Here n counts qubits and N is matrix dimension ($N = 2^n$ for a circuit). The survey's ~ includes consequences whose gate-set transfer is not formalized in the corpus; a plain check does not assert QCert verification. Imaginary-family checks refer to new computer-assisted proofs [56]; their complete Lean checks cover only $N = 2, 3$ and are outside this corpus. The higher-dimensional Clifford+T matrix presentation remains incomplete. L_N is defined in Table 1; the other notation is as in Figure 2.

4.1 Comparison of the Agda and Lean Formalizations

The Lean [29] version formalizes the same sixteen results using Mathlib's standard definitions [64], with matrix groups over real or complex numbers. It translates the Agda framework, so agreement between the two does not independently validate their shared design. Figure 7 compares proof size, verification time, and execution time. Lean developments generally use fewer lines and Agda records lower verification times in thirteen rows, but proofs and measurement methods differ. Translating the same proof, for Toffoli–Hadamard and for the four-bit circuits of Section 6, yields similar amounts of text, and Lean checks the translation in 3.3 to 3.4 times Agda's time. Agda provides executable normal forms, whereas Lean's are existence results or `noncomputable` definitions. Panel (c) measures their executable circuit-equivalence procedures, where Lean is faster

from three qubits onward. Appendix C gives the detailed comparison and measurement methods, summarized in Table 2.

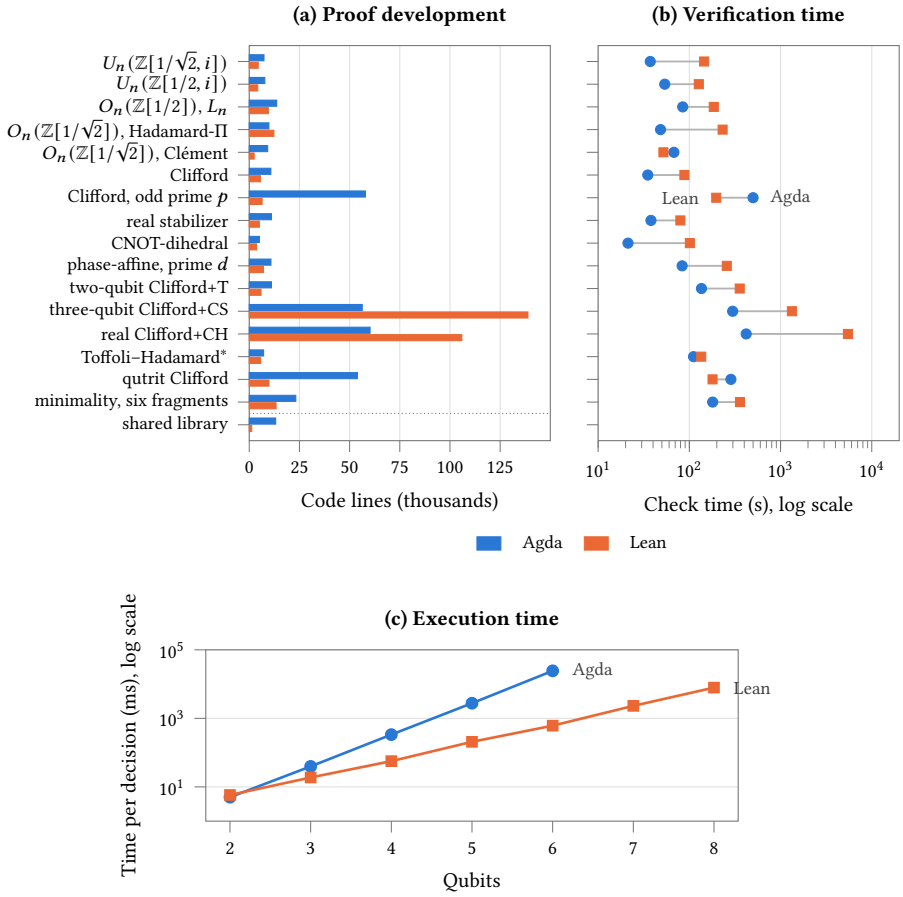


Fig. 7. The Agda and Lean formalizations of each row of Table 1. (a) Code lines, excluding blank lines and comments. For Toffoli–Hadamard* it shows another pair, one proof in both assistants (Appendix C): a second Agda proof and its translation to Lean. The last row is the shared library of each framework. (b) The check times of Table 1, Agda (circles) and Lean (squares), measured on the same machine, with methods described in Appendix C. (c) The time each compiled formalization takes to decide that two Clifford circuits of 100 gates have the same matrix, by the procedure its soundness proof uses, as medians over three runs of five random circuits.

5 Corrections to the Source Papers

The formalizations in Table 1 identify two kinds of issues in the source papers. Most are typos, and some of them appear in statements that the proofs rely on. These are easy to fix. Two others are harder to repair. One is a gap in a proof, and the other is a relation missing from a theorem. We describe both below.

The gap is in the proof of Theorem 4.14 of Hadamard-Π [31], which states that the relations in its Figure 6, over the one- and two-level matrices $(-1)_{[a]}$, $X_{[a,b]}$, and $H_{[a,b]}$, present the group

$O_n\left(\mathbb{Z}\left[1/\sqrt{2}\right]\right)$. One case of the proof relies on Lemmas A.16 and A.17, which have counterexamples in dimension 6, and the case analysis built on them misses a matrix in dimension 9. Their main result is however still correct: Clément's unpublished notes [23] present the same group with different relations, and his case analysis covers the missing case, using which the formalization of Theorem 4.14 is completed. We also prove that the two sets of relations are equivalent in every dimension and formalize Clément's proof [5].

The relation is missing from Theorem 5.2 of [3]. It states that the relations in its Figures 4 and 7, over the Toffoli and $\mathrm{K} = \mathrm{H} \otimes \mathrm{H}$ gates, present the group $O_8\left(\mathbb{Z}\left[1/2\right]\right)$. This is the first of two steps towards presenting the scaled dyadic orthogonal group L_8 by three-qubit Toffoli–Hadamard circuits. The relation r_{K} in its Section 5.5, which splits K on two qubits into two copies controlled by the third qubit, is in neither figure. Nor can it be derived from them. Hence, Theorem 5.2 fails as stated. Our formalization [6] shows that r_{K} is independent by giving an interpretation of circuits that every other relation preserves but r_{K} does not, and proves the presentations of $O_8\left(\mathbb{Z}\left[1/2\right]\right)$ and L_8 with r_{K} restored.

6 A New Result: Complete Equations for Four-Bit Reversible Circuits

Beyond the published results, we use QCert to prove and check a new completeness theorem for classical reversible circuits. The gates X (NOT), CX (CNOT), and CCX (Toffoli) generate the reversible circuits on bits. Without ancillas, circuits on $k \leq 3$ wires realize every permutation of the 2^k bit strings, while on $k \geq 4$ wires every gate is an even permutation and the circuits realize exactly the even ones [61]. Complete equational theories are known for linear reversible circuits [43, 58] and for circuits over multiple-control Toffoli gates [32, 33]. We give one for the gate set $\{\mathrm{X}, \mathrm{CX}, \mathrm{CCX}\}$ on at most four wires, the theory $\mathcal{T}$ of the seven equations in Figure 8. Equations (T1)–(T6) act on at most three wires and (T7) on four. Wires are numbered from 0 at the bottom, as in the rest of QCert (Appendix B.1). A circuit is a product of gates and of swaps of adjacent wires, written in the order in which they are applied. Two circuits are *equal modulo* $\mathcal{T}$ when one can be rewritten into the other by the seven equations and the structural rules of Figure 9, each of which may be used in either direction.

A *state* of k wires is a string of k bits, and a circuit u on k wires denotes a permutation $[\![u]\!]$ of the 2^k states. Write G_k for the circuits on k wires modulo $\mathcal{T}$, with the product of circuits.

Theorem 6.1 (soundness and completeness). *On any number of wires, two circuits that are equal modulo* $\mathcal{T}$ *denote the same permutation. Conversely, for* $1 \leq k \leq 4$*, two circuits on* k *wires that denote the same permutation are equal modulo* $\mathcal{T}$*. For these* k*,* G_k *is a group, and* $u \mapsto [\![u]\!]$ *is an isomorphism from* G_k *onto* $\mathrm{Sym}(2^k)$ *for* $1 \leq k \leq 3$ *and from* G_4 *onto* $\mathrm{Alt}(16)$*.*

On four wires, every circuit also has an explicit normal form. Let the *halves* H_0 and H_1 be the sets of states in which wire 3 is 0 and 1. Each half is a copy of the eight states of the lower wires 0, 1, and 2, and $\mathrm{Sym}(8)$ and $\mathrm{Alt}(8)$ are the groups of all permutations and of the even permutations of these eight states. A permutation of the sixteen states that preserves the halves acts on H_0 and on H_1 as two elements of $\mathrm{Sym}(8)$, its *blocks*. We also write a circuit for its permutation, and we write products of permutations in the order of circuits, so pq applies p and then q. The development fixes three families of circuits (Appendix E.12). For a set S of eight states, the circuit r_S maps S onto H_0. For $B \in \mathrm{Sym}(8)$, the circuit $\mathrm{D}(B)$ acts on the lower wires and has the blocks B and B. For $E \in \mathrm{Alt}(8)$, the circuit $\mathrm{C}(E)$ has the blocks the identity and E. The *block normal form* with the *coordinates* (E, B, S) is the circuit $\mathrm{NF}(E, B, S) = r_S\,\mathrm{D}(B)\,\mathrm{C}(E)$. It is r_S, which maps S onto H_0, followed by a circuit with the blocks B and BE.

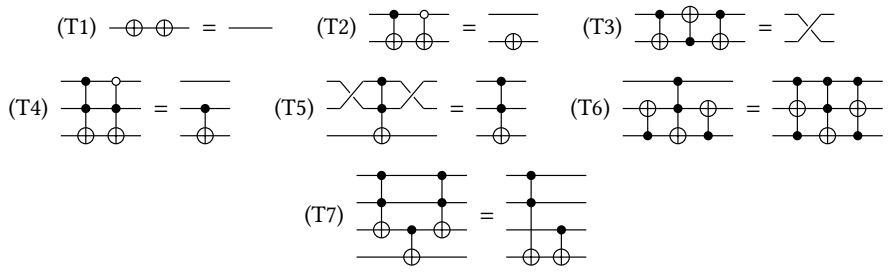


Fig. 8. The theory $\mathcal{T}$ for circuits over $\{\mathrm{X}, \mathrm{CX}, \mathrm{CCX}\}$. Time runs left to right, and wire 0 is the bottom wire. A filled control tests 1, an open control tests 0 (it is the filled one conjugated by X on its wire), $\oplus$ is a NOT target, and crossed wires are the structural swap. A CNOT or Toffoli drawn in another position, as in (T3), (T6), and (T7), is the gate on adjacent wires with its target below its controls, conjugated by swaps. Each equation can be used in either direction and, by the structural rules of Figure 9, on any wires of a circuit, adjacent or not and in any order.

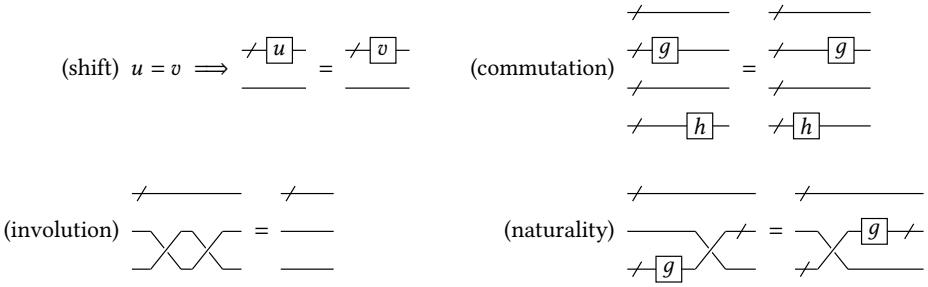


Fig. 9. The structural rules for circuits over $\{\mathrm{X}, \mathrm{CX}, \mathrm{CCX}\}$, which QCert supplies for every gate set with a structural swap (Appendix B.1). Here u and v are circuits, g and h are X, CX, CCX, or the swap, and a slashed wire carries the wires of the box on it or, if it has none, any number of idle wires. An equation also holds with an idle wire below it, a gate commutes with any gate above it, the swap is an involution, and a gate slides past a wire that crosses all of its wires; for g the swap, (naturality) is the braid relation. (shift) adds wires only below, while the other rules hold under any idle wires.

Theorem 6.2 (block normal form). *Let u and w be four-wire circuits.*

(1) *Let S be the set of states that u maps into H_0, let B_0 and B_1 be the blocks of $r_S^{-1}u$, and let $E = B_0^{-1}B_1$. Then E is even, and u is equal modulo $\mathcal{T}$ to $\mathrm{NF}(E, B_0, S)$.*
(2) *Two normal forms that denote the same permutation have the same coordinates, and every even permutation of the sixteen states is denoted by a normal form. Hence u is equal modulo $\mathcal{T}$ to exactly one normal form, and $(E, B, S) \mapsto [\![\mathrm{NF}(E, B, S)]\!]$ is a bijection from $\mathrm{Alt}(8) \times \mathrm{Sym}(8) \times \binom{\mathbb{F}_2^4}{8}$ onto $\mathrm{Alt}(16)$, where $\binom{\mathbb{F}_2^4}{8}$ is the set of sets of eight states.*
(3) *For any coordinates (E, B, S), let S' be the set of states that w maps into S, and let A_0 and A_1 be the blocks of $r_{S'}^{-1}\, w\, r_S$. Then $w\,\mathrm{NF}(E, B, S)$ is equal modulo $\mathcal{T}$ to $\mathrm{NF}(E', B', S')$, where $B' = A_0 B$ and $B'E' = A_1 BE$.*

Both theorems are proved in Agda [4], and the proof is translated to Lean module for module (Appendix C). Statements (6) to (10) are five statements of the Agda MainTheorem module, which also states completeness and the presentations of $\mathrm{Sym}(2^k)$ on fewer wires, and the second item of Theorem 6.2 (unique and onto). Here $[\![w]\!]$ is the function of w on bit vectors, $\doteq$ is pointwise equality,

$u \bullet w$ applies u and then w, coords u is the triple of coordinates of the first item, and absorb c w that of the third.

```
sound : ∀ {n} {w v : Circuit n} → ⊢ w ≈ v → [[ w ]] ≐ [[ v ]]                (6)
complete : {w v : Circuit 4} → [[ w ]] ≐ [[ v ]] → ⊢ w ≈ v                   (7)
presentation₄ : (4 CRel,_===_) IsPresentationOf Alt-group 16                 (8)
normal-form : (u : Circuit 4) → ⊢ u ≈ NF (coords u)                          (9)
absorption : (c : Coords) (w : Circuit 4) → ⊢ w • NF c ≈ NF (absorb c w)     (10)
```

Proof idea. The proof follows the second strategy of Figure 3, with presentations of permutation groups as the complete rule sets. One wire is immediate from (T1). On two and three wires, we number the states along a Gray code, so that the transposition of two consecutive states is a single gate with chosen control polarities, a CNOT on two wires and a Toffoli gate on three. From (T1)–(T6) we derive the Coxeter relations of the symmetric group for these gates and write every gate as a product of them, and completeness follows from the Coxeter presentation [28]. On four wires such a transposition is an odd permutation, hence not a circuit. Instead, the Reidemeister–Schreier method [48], which the framework provides as a verified theorem (Section 3.2), turns the Coxeter presentation of Sym(16) into a presentation of Alt(16) whose generators are products of two transpositions. Read as an equation between circuits, each of its relations has gates that leave one wire untargeted. This reduces completeness to the circuits whose gates never target wire 3. Such a circuit is a product of *controlled matchings* followed by a circuit on the lower wires, where a controlled matching is the CNOT from wire 3 onto wire 0 conjugated by a circuit on the lower wires. The circuits on the lower wires are settled by completeness on three wires, and the controlled matchings by the presentation of Alt(8) obtained in the same way, from three relations between them. Equation (T7) enters only in this last part, through four-wire derivations that a search finds and Agda checks; the longest takes 496 steps. The normal form is not used to prove completeness. Theorem 6.2 follows from Theorem 6.1, because the coordinates of a circuit are read off its permutation. Appendix E gives the proofs. The formalization has 40 modules and about 13,000 lines. Given the shared library, Agda checks it in 85 seconds, and Lean checks its translation in 277.

7 Discussion and Future Work

We formalize the results of sixteen papers in QCert, using consistent circuit syntax, a unified definition of group presentations, and shared libraries for rings and their matrix groups. Since all formalizations use these definitions, presentations of the same group from different papers can be compared and proved equivalent. The same libraries can be reused for new algorithms, and the induction that turns an exact synthesis algorithm into a completeness proof carries over with little change. The main part specific to each algorithm is the case analysis over matrices and generators. As the most error-prone part of the proof, it is where machine checking in QCert matters most.

Reusing this infrastructure, we have obtained complete equational theories for circuits over $\{X, CX, CCX\}$ and for Toffoli–Hadamard circuits. We are now preparing mathematical expositions of these results.

At the moment, not every completeness result fits into QCert, as its scope is limited in two ways. First, it supports only discrete gate sets, so it does not cover completeness results for circuits with continuous parameters, such as rotations by arbitrary angles [25–27]. Second, control is not a circuit constructor in QCert, so it does not cover results that treat control as a primitive operation on circuits [16, 30, 59].

We plan to extend the infrastructure of QCert to more families of quantum circuits, including the two above. Completeness is still open for many discrete gate sets in higher-dimensional quantum systems, and their case analyses are often too large to check by hand. In particular, we plan to study the completeness of Clifford circuits in all dimensions with the help of QCert. Beyond completeness, the exact synthesis libraries can be reused for T-count optimization of multi-qubit Clifford+T circuits.

References

[1] Matthew Amy, Jianxin Chen, and Neil J. Ross. 2018. A finite presentation of CNOT-dihedral operators. In *Proceedings of the 14th International Conference on Quantum Physics and Logic (QPL 2017) (Electronic Proceedings in Theoretical Computer Science, Vol. 266)*. 84–97. doi:10.4204/EPTCS.266.5

[2] Matthew Amy, Andrew N. Glaudell, and Neil J. Ross. 2020. Number-theoretic characterizations of some restricted Clifford+T circuits. *Quantum* 4 (2020), 252.

[3] Matthew Amy, Neil J. Ross, and Scott Wesley. 2024. A sound and complete equational theory for 3-qubit Toffoli-Hadamard circuits. In *Proceedings of the 21st International Conference on Quantum Physics and Logic (QPL 2024) (Electronic Proceedings in Theoretical Computer Science, Vol. 406)*. 1–43. arXiv:2407.11152 doi:10.4204/EPTCS.406.1

[4] Anonymous. 2026. QCert: Complete equations for four-bit reversible circuits, formalized in Agda and Lean. Supplementary material, module `XCXCCX-4bit/MainTheorem` (completeness on one to four wires, the presentations of $\mathrm{Sym}(2^k)$ and $\mathrm{Alt}(16)$, and the block normal form), and its Lean translation `QCert/XCXCCX4bit/MainTheorem`.

[5] Anonymous. 2026. QCert: The presentations of $O_n(\mathbb{Z}[1/\sqrt{2}])$ by Hadamard-Π and by Clément, formalized in Agda. Supplementary material, modules `Real-Clifford+CH-TwoLevel/Equivalence` (the equivalence of the two sets of relations) and `Real-Clifford+CH-TwoLevel-Clement` (Clément's proof and its decision trees).

[6] Anonymous. 2026. QCert: Three-qubit Toffoli–Hadamard circuits with the relation r_{K}, formalized in Agda. Supplementary material, modules `CCX+H-3qubit-RS/Theorems` (the presentations of $O_8(\mathbb{Z}[1/2])$ and L_8) and `CCX+H-3qubit-RS/Independence` (the interpretation showing that r_{K} is not derivable).

[7] Anthropic. 2026. Formalizing Fermat's Last Theorem. Research post. https://www.anthropic.com/research/formalizing-fermats-last-theorem 4 September 2026.

[8] Alexander Bastounis, Fabian Circelli, and Anders C. Hansen. 2026. Navier-Stokes lost in translation: Why Lean verification of AI autoformalisation does not guarantee correct natural language proofs. *arXiv preprint arXiv:2610.08144* (2026).

[9] Maor Ben-Shahar, Jonathan Morag, Dirk Englund, and Kfir Sulimany. 2026. Quantum cryptography and autoformalization in Lean. arXiv:2610.17286 [quant-ph] doi:10.48550/arXiv.2610.17286

[10] Xiaoning Bian and Yuan Feng. 2027. Normal forms and complete relations for circuits in Agda. *Proceedings of the ACM on Programming Languages* POPL (2027). To appear.

[11] Xiaoning Bian, Sarah Meng Li, Neil J. Ross, John van de Wetering, and Yuming Zhao. 2026. A complete and natural rule set for multi-qudit Clifford circuits in all odd prime dimensions. https://github.com/onestruggler/acir/releases/tag/qip-submission

[12] Xiaoning Bian, Sarah Meng Li, Neil J. Ross, John van de Wetering, and Yuming Zhao. 2026. A complete and natural rule set for multi-qudit Clifford circuits in all odd prime dimensions. arXiv:2609.40106

[13] Xiaoning Bian and Peter Selinger. 2021. Generators and relations for $U_n(\mathbb{Z}[1/2, i])$. In *Proceedings of the 18th International Conference on Quantum Physics and Logic (QPL 2021) (Electronic Proceedings in Theoretical Computer Science, Vol. 343)*. 145–164. arXiv:2105.14047 doi:10.4204/EPTCS.343.8

[14] Xiaoning Bian and Peter Selinger. 2023. Generators and relations for 2-qubit Clifford+T operators. In *Proceedings of the 19th International Conference on Quantum Physics and Logic (QPL 2022) (Electronic Proceedings in Theoretical Computer Science, Vol. 394)*. 13–28. arXiv:2204.02217 doi:10.4204/EPTCS.394.2

[15] Xiaoning Bian and Peter Selinger. 2023. Generators and relations for 3-qubit Clifford+CS operators. In *Proceedings of the 20th International Conference on Quantum Physics and Logic (QPL 2023) (Electronic Proceedings in Theoretical Computer Science, Vol. 384)*. 114–126. arXiv:2306.08530 doi:10.4204/EPTCS.384.7

[16] Colin Blake. 2026. A complete equational presentation of qudit circuits via polycontrolled PROPs. In *Proceedings of the 51st International Symposium on Mathematical Foundations of Computer Science (MFCS 2026) (Leibniz International Proceedings in Informatics, Vol. 386)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 6:1–6:18. arXiv:2602.09873 doi:10.4230/LIPIcs.MFCS.2026.6

[17] Colin Blake. 2026. Completeness for prime-dimensional phase-affine circuits. arXiv:2603.06466

[18] Colin Blake. 2026. Simpler presentations for many fragments of quantum circuits. In *Proceedings of the 11th International Conference on Formal Structures for Computation and Deduction (FSCD 2026) (Leibniz International Proceedings in*

Informatics, Vol. 378). 6:1–6:20. arXiv:2602.09874 doi:10.4230/LIPIcs.FSCD.2026.6

[19] Ana Bove, Peter Dybjer, and Ulf Norell. 2009. A brief overview of Agda – a functional language with dependent types. In *Theorem Proving in Higher Order Logics (TPHOLs 2009) (Lecture Notes in Computer Science, Vol. 5674)*. Springer, Berlin, Heidelberg, 73–78.

[20] Jacques Carette, Chris Heunen, Robin Kaarsgaard, and Amr Sabry. 2024. With a few square roots, quantum computing is as easy as Π. *Proceedings of the ACM on Programming Languages* 8, POPL (2024). arXiv:2310.14056

[21] Christophe Chareton, Sébastien Bardin, François Bobot, Valentin Perrelle, and Benoît Valiron. 2021. An automated deductive verification framework for circuit-building quantum programs. In *Programming Languages and Systems: 30th European Symposium on Programming (ESOP 2021) (Lecture Notes in Computer Science, Vol. 12648)*. 148–177.

[22] Vikraman Choudhury, Jacek Karwowski, and Amr Sabry. 2022. Symmetries in reversible programming: From symmetric rig groupoids to reversible programming languages. *Proceedings of the ACM on Programming Languages* 6, POPL, Article 6 (2022), 32 pages.

[23] Alexandre Clément. 2019. A complete set of relations for the real Clifford+T group. Unfinished manuscript. https://www.mathstat.dal.ca/~aclement/Case%20distinction.pdf

[24] Alexandre Clément. 2026. A complete equational theory for real-Clifford+CH quantum circuits. arXiv:2602.06644

[25] Alexandre Clément, Noé Delorme, and Simon Perdrix. 2024. Minimal equational theories for quantum circuits. In *Proceedings of the 39th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2024)*. 27:1–27:14. arXiv:2311.07476 doi:10.1145/3661814.3662088

[26] Alexandre Clément, Noé Delorme, Simon Perdrix, and Renaud Vilmart. 2024. Quantum circuit completeness: Extensions and simplifications. In *Proceedings of the 32nd EACSL Annual Conference on Computer Science Logic (CSL 2024) (Leibniz International Proceedings in Informatics, Vol. 288)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 20:1–20:23. arXiv:2303.03117 doi:10.4230/LIPIcs.CSL.2024.20

[27] Alexandre Clément, Nicolas Heurtel, Shane Mansfield, Simon Perdrix, and Benoît Valiron. 2023. A complete equational theory for quantum circuits. In *Proceedings of the 38th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2023)*. 1–13. arXiv:2206.10577 doi:10.1109/LICS56636.2023.10175801

[28] H. S. M. Coxeter and W. O. J. Moser. 1980. *Generators and Relations for Discrete Groups* (4th ed.). Ergebnisse der Mathematik und ihrer Grenzgebiete, Vol. 14. Springer, Berlin. doi:10.1007/978-3-662-21943-0

[29] Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28 (Lecture Notes in Computer Science, Vol. 12699)*. Springer, 625–635. doi:10.1007/978-3-030-79876-5_37

[30] Noé Delorme and Simon Perdrix. 2026. Diagrammatic reasoning with control as a constructor, applications to quantum circuits. In *Foundations of Software Science and Computation Structures (FoSSaCS 2026) (Lecture Notes in Computer Science, Vol. 16503)*. Springer Nature Switzerland, Cham, Switzerland, 240–261. arXiv:2508.21756 doi:10.1007/978-3-032-22730-0_12

[31] Wang Fang, Chris Heunen, and Robin Kaarsgaard. 2026. Hadamard-Π: Equational quantum programming. *Proceedings of the ACM on Programming Languages* 10, POPL, Article 5 (2026), 27 pages. arXiv:2506.06835 doi:10.1145/3776647

[32] Shiguang Feng and Lvzhou Li. 2026. A complete set of transformation rules for reversible circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 45, 8 (2026), 3711–3724. arXiv:2508.17273 doi:10.1109/TCAD.2025.3641533

[33] Shiguang Feng, Zongxing Xiong, and Lvzhou Li. 2026. On the completeness of transformation rules in reversible logic synthesis. arXiv:2607.19059

[34] Brett Giles and Peter Selinger. 2013. Exact synthesis of multiqubit Clifford+T circuits. *Physical Review A* 87 (2013), 032332. arXiv:1212.0506

[35] Andrew N. Glaudell, Neil J. Ross, and Jacob M. Taylor. 2021. Optimal two-qubit circuits for universal fault-tolerant quantum computation. *npj Quantum Information* 7 (2021), 103. arXiv:2001.05997 doi:10.1038/s41534-021-00424-z

[36] Seth Eveson Murray Greylyn. 2014. *Generators and relations for the group* $U_4(\mathbb{Z}[1/\sqrt{2}, i])$. MSc thesis. Dalhousie University. arXiv:1408.6204

[37] David Gross. 2006. Hudson's theorem for finite-dimensional quantum systems. *J. Math. Phys.* 47, 12 (2006), 122107. doi:10.1063/1.2393152

[38] Shamgar Gurevich and Ronny Hadani. 2007. The geometric Weil representation. *Selecta Mathematica* 13 (2007), 465–481. doi:10.1007/s00029-007-0047-3

[39] Shamgar Gurevich and Ronny Hadani. 2010. Bounds on certain higher-dimensional exponential sums via the self-reducibility of the Weil representation. arXiv:math/0612765 [math.RT] https://arxiv.org/abs/math/0612765

[40] Thomas Hales. 2026. What mathematicians should know about the Lean theorem prover: Questions of reliability and AI. Guest post on Terence Tao's blog *What's new*. https://terrytao.wordpress.com/2026/10/09/what-mathematicians-should-know-about-the-lean-theorem-proverquestions-of-reliability-and-ai/ 9 October 2026.

[41] Kesha Hietala, Robert Rand, Shih-Han Hung, Xiaodi Wu, and Michael Hicks. 2021. A verified optimizer for quantum circuits. *Proceedings of the ACM on Programming Languages* 5, POPL, Article 37 (2021), 29 pages. doi:10.1145/3434318

[42] Kazumi Kasaura, Kei Tsukamoto, Kento Mori, Risa Mizuno, Takahiro Namatame, Yuta Oriike, Masaya Taniguchi, Sho Sonoda, and Hayata Yamasaki. 2026. Lean-Quantum: Toward AI-assisted formalization of quantum information. arXiv:2607.05492 [quant-ph] doi:10.48550/arXiv.2607.05492

[43] Yves Lafont. 2003. Towards an algebraic theory of Boolean circuits. *Journal of Pure and Applied Algebra* 184, 2–3 (2003), 257–310. doi:10.1016/S0022-4049(03)00069-0

[44] Sarah Meng Li, Michele Mosca, Neil J. Ross, John van de Wetering, and Yuming Zhao. 2025. A complete and natural rule set for multi-qutrit Clifford circuits. In *Proceedings of the 22nd International Conference on Quantum Physics and Logic (QPL 2025) (Electronic Proceedings in Theoretical Computer Science, Vol. 426)*. 23–78. arXiv:2508.14670 doi:10.4204/EPTCS.426.2

[45] Sarah Meng Li, Neil J. Ross, and Peter Selinger. 2021. Generators and relations for the group $O_n(\mathbb{Z}[1/2])$. In *Proceedings of the 18th International Conference on Quantum Physics and Logic (QPL 2021) (Electronic Proceedings in Theoretical Computer Science, Vol. 343)*. 210–264. arXiv:2106.01175 doi:10.4204/EPTCS.343.11

[46] Sirui Lu, Ruixuan Deng, Yanqiao Zhu, and Zhengfeng Ji. 2026. Long-horizon autoformalization of a core theorem underlying MIP* = RE. arXiv:2609.19814 [quant-ph] doi:10.48550/arXiv.2609.19814

[47] Saunders Mac Lane. 1965. Categorical algebra. *Bull. Amer. Math. Soc.* 71, 1 (1965), 40–106. doi:10.1090/S0002-9904-1965-11234-4

[48] Wilhelm Magnus, Abraham Karrass, and Donald Solitar. 2004. *Combinatorial group theory: Presentations of groups in terms of generators and relations* (2nd revised ed.). Dover Publications.

[49] Justin Makary, Neil J. Ross, and Peter Selinger. 2021. Generators and relations for real stabilizer operators. In *Proceedings of the 18th International Conference on Quantum Physics and Logic (QPL 2021) (Electronic Proceedings in Theoretical Computer Science, Vol. 343)*. 14–36. arXiv:2109.05655 doi:10.4204/EPTCS.343.2

[50] Dmitri Maslov, Gerhard W. Dueck, D. Michael Miller, and Camille Negrevergne. 2008. Quantum circuit simplification and level compaction. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 27, 3 (2008), 436–444. doi:10.1109/TCAD.2007.911334

[51] Yunseong Nam, Neil J. Ross, Yuan Su, Andrew M. Childs, and Dmitri Maslov. 2018. Automated optimization of large quantum circuits with continuous parameters. *npj Quantum Information* 4 (2018), 23. doi:10.1038/s41534-018-0072-4

[52] Markus Neuhauser. 2002. An Explicit Construction of the Metaplectic Representation over a Finite Field. *Journal of Lie Theory* 12, 1 (2002), 15–30. doi:10.5802/jolt.250

[53] Ulf Norell. 2007. *Towards a practical programming language based on dependent type theory*. Ph. D. Dissertation. Chalmers University of Technology.

[54] OpenAI. 2026. Mathematical manuscripts and Lean formalizations from an internal OpenAI model. GitHub repository. https://github.com/openai/math Released 6 October 2026.

[55] Jennifer Paykin, Robert Rand, and Steve Zdancewic. 2017. QWIRE: A core language for quantum circuits. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages (POPL 2017)*. 846–858.

[56] RelationsStatus 2026. Complete relations for restricted Clifford+T operators: status by exact synthesis algorithm. Unpublished working notes. Version of 7 November 2026.

[57] Yuanjie (Collin) Ren, Zhicheng Zhang, Junyi Liu, Jinzheng Li, Yidi Qi, and Mingsheng Ying. 2026. QEDIA: Quantum program verification in Lean. arXiv:2610.17708 [quant-ph] doi:10.48550/arXiv.2610.17708

[58] Issei Sakashita. 2013. A complete transformation rule set and a minimal equation set for CNOT-based 3-qubit quantum circuits (Draft). arXiv:1304.5916

[59] William Schober and Scott Wesley. 2026. A complete equational theory for quantum circuits with generalized control. https://cqi.inf.usi.ch/publications/Hierarchical_QPL2026.pdf Accepted to the proceedings track of the 23rd International Conference on Quantum Physics and Logic (QPL 2026).

[60] Peter Selinger. 2015. Generators and relations for n-qubit Clifford operators. *Logical Methods in Computer Science* 11, 2:10 (2015). arXiv:1310.6813 doi:10.2168/LMCS-11(2:10)2015

[61] Vivek V. Shende, Aditya K. Prasad, Igor L. Markov, and John P. Hayes. 2003. Synthesis of reversible logic circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 22, 6 (2003), 710–722. doi:10.1109/TCAD.2003.811448

[62] Sergey Shirobokov, Ben Jaderberg, Katerina Hristova, George Tsoukalas, Johannes Bausch, Eric Wieser, Francisco J. H. Heras, Nicholas C. Rubin, Guang Hao Low, Sébastien Racanière, Nenad Tomašev, Lei M. Zhang, James Watson, Tanuj Khattar, Aws Albarghouthi, Stavan Jain, Harun Khan, Leopoldo Sarra, Andrew W. Senior, Michael Broughton, Hussain Masoom, Sergio Boixo, Ryan Babbush, Swarat Chaudhuri, and Pushmeet Kohli. 2026. Discovering formally verified quantum programs with AI. arXiv:2611.09822 [quant-ph] doi:10.48550/arXiv.2611.09822

[63] Seyon Sivarajah, Silas Dilkes, Alexander Cowtan, Will Simmons, Alec Edgington, and Ross Duncan. 2020. t|ket⟩: A retargetable compiler for NISQ devices. *Quantum Science and Technology* 6, 1 (2020), 014003. doi:10.1088/2058-9565/ab8e92

[64] The mathlib Community. 2020. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*. 367–381. doi:10.1145/3372885.3373824

[65] Mingkuan Xu, Zikun Li, Oded Padon, Sina Lin, Jessica Pointing, Auguste Hirth, Henry Ma, Jens Palsberg, Alex Aiken, Umut A. Acar, and Zhihao Jia. 2022. Quartz: Superoptimization of quantum circuits. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI 2022)*. Association for Computing Machinery, New York, NY, USA, 625–640. doi:10.1145/3519939.3523433

[66] Shigeru Yamashita and Igor L. Markov. 2010. Fast equivalence-checking for quantum circuits. *Quantum Information and Computation* 10, 9–10 (2010), 721–734. doi:10.26421/QIC10.9-10-1

[67] Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. 2023. LeanDojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*.

[68] Lei Zhang, Mingrui Jing, Yusheng Zhao, Jiahong Liu, and Xin Wang. 2026. Lean-QuantumAlg: A framework for AI agents to produce quantum algorithms that humans can trust. arXiv:2610.30216 [quant-ph] doi:10.48550/arXiv.2610.30216

[69] Li Zhou, Gilles Barthe, Pierre-Yves Strub, Junyi Liu, and Mingsheng Ying. 2023. CoqQ: Foundational verification of quantum programs. *Proceedings of the ACM on Programming Languages* 7, POPL, Article 29 (2023), 33 pages.

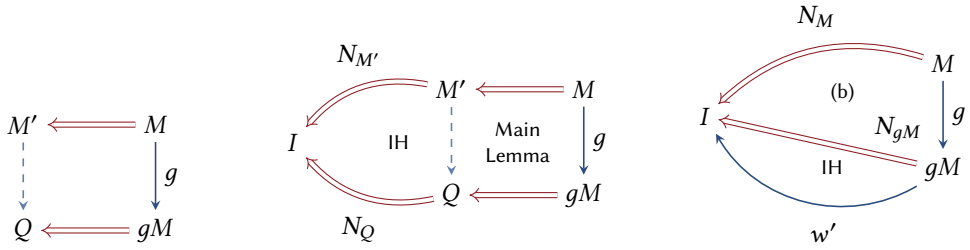
[70] Chengkai Zhu, Ziao Tang, Guocheng Zhen, Yimeng Cao, Yusheng Zhao, Ranyiliu Chen, Xuanqiang Zhao, Lei Zhang, and Xin Wang. 2026. Lean-QIT: Towards a formal infrastructure for quantum information theory. arXiv:2607.09632 [quant-ph] doi:10.48550/arXiv.2607.09632

A Completeness from Exact Synthesis

For the matrix groups discussed in Section 2, normal forms come from exact synthesis, and the proof that every circuit can be transformed into its normal form has the same structure for each algorithm. Each matrix, or *state*, has a *level*, built from quantities such as the least denominator exponent of a column, and the algorithm reduces each state M to the identity in steps that lower the level. These steps form the normal word N_M.

The *Main Lemma* says that for every state M and generator g, a diagram of relations closes the square formed by g and the algorithm's next steps from M and gM, passing only through states of lower level (Figure 10a).

By well-founded induction on the level, the relations then rewrite $N_{gM}\, g$ into N_M (Figure 10b), and by induction on length, every word that reduces M to the identity rewrites into N_M (Figure 10c). Two words with the same matrix therefore rewrite into each other. These inductions have the same form for every presentation, so the main work in each formalization is its Main Lemma.



(a) The Main Lemma. (b) Induction on the level. (c) Induction on length.

Fig. 10. Completeness from the Main Lemma. Plain arrows are words, double arrows are steps or runs of the algorithm, and dashed arrows are words through states of lower level.

B QCert Implementation Details

This appendix gives the circuit encoding, theorem notation, and matrix equality procedure used by the framework in Section 3.

B.1 Wire Placement and Structural Rules

Wires are numbered from 0 at the bottom, and a gate acts on the bottom wires of a circuit. A shift places a circuit on n wires onto wires $1, \ldots, n$ of a circuit with $n + 1$ wires. In the matrix semantics, both placing a gate in a larger circuit and shifting a circuit are Kronecker products with an identity. This defines wire placement once for all gate sets.

Each source relation is stated on as many wires as the paper draws it on. Structural rules allow an equation to be shifted by one wire, gates on disjoint wires to commute, and a scalar to be placed on any wire. QCert extends the earlier framework's one- and two-wire syntax with three-wire gates and their structural rule. Scalars are zero-wire generators; their placement rule allows exact equations to account for global phase.

In the earlier framework, swap is an ordinary gate whose laws must be included in each rule set. QCert also supports a PROP presentation, in which a designated two-wire gate is the swap and the framework supplies its involution law and the laws for moving gates across it. The source rule set then contains only its own relations, each on its original wires; swap and its laws are supplied by the framework. We use this representation for the circuits of [1, 17, 18], and for the reversible circuits of Section 6, whose structural rules are drawn in Figure 9.

B.2 Theorem Notation

The signatures in Section 3 use `n CRel._===_` for relations on circuits with n wires and `n MRel._===_` for relations on words over multi-level matrices of dimension n. In Statement (1), the former contains R1–R16 of Figure 1 of [49], lifted to n wires. In Statement (2), the latter contains the relations of Figure 6 of Hadamard-Π [31].

The judgement `⊢ w ≈ v` states that w and v are equal by the relations. The qubit developments interpret matrices in $\mathbb{D}[\omega]$ and use $_\approx^{\mathsf{U}}_$ for their equality; the qudit developments compare interpretations in their own groups. The `IsSubPresentationOf` record contains the semantics and its soundness and completeness proofs, and `IsPresentationOf` adds surjectivity.

Minimality is stated for the rule set `n ⊢_===_` in Statement (5). Unlike the generated equivalence relation, this retains the individual rules, so removing one is defined. The predicate `IsMinimal` therefore takes this rule set rather than its generated equivalence relation.

B.3 Matrix Equality Checking

The main computational cost came from Agda's conversion checker. It accepts syntactically identical terms immediately; otherwise, it evaluates them to compare their results. For long matrix products, this evaluation can take exponential time. We keep circuit interpretations abstract and decide matrix equality by computing a single Boolean using row operations. For these products, this is orders of magnitude faster than comparing matrices entry by entry. These techniques are included in the agents' instructions so that later formalizations avoid the same slowdowns.

C Details of the Agda and Lean Comparison

The Lean [29] version of QCert formalizes the same sixteen results on top of Mathlib [64]. Once the Agda formalization exists, translating it to Lean is comparatively easy, so the Lean version is not a second achievement of the same kind. Its point is its theorem statements. They rest on Mathlib's standard definitions, with matrix groups over Mathlib's real or complex numbers in place of our exact rings, so a reader can check them against the source papers using definitions that are already widely used and reviewed. For qudit Clifford, the Lean statement also removes an argument that the Agda statement leaves to external mathematics (Section D.3).

Nor is the Lean framework an independent implementation. We instructed the coding agent that wrote it not to consult our Agda implementation, but the Lean framework is nonetheless a translation of the Agda one. Its words, circuit generators, structural rules, and presentation and sub-presentation predicates are those of the Agda framework, down to the names of the definitions and the wording of their documentation. The two formalizations therefore share the design of their framework, and would share any mistake in it, so their agreement is weaker evidence than two independent formalizations would give. They use different representations and sometimes different proofs, so the theorem statements of each must be checked against the source papers separately. We compare their definitions, proof development, verification time, and execution time; Table 2 summarizes the comparison. These observations concern our two formalizations, not Agda and Lean in general.

Definitions and Semantics. The translation keeps the definitions but adapts their representation to Mathlib. Agda uses its own word type, with explicit brackets, and the setoid-based algebra of the standard library. A presentation is a record containing a semantics and a group isomorphism, or a monomorphism for a sub-presentation. Lean uses Mathlib's free monoid and the monoid congruence generated by the relations. Its presentations are predicates in `Prop`, from which it

	Agda	Lean
Proof development		
Logic	constructive: every module `--safe`, no postulates	classical: every main theorem uses `Classical.choice`
Library	standard library (104 modules imported), EucDomain	Mathlib (56 modules imported directly)
Proofs	terms, by pattern matching and equational reasoning; reflection-based solvers	tactics: about 75,000 tactic blocks, 69,000 automation calls
Reused from the sources	the authors' Agda proofs for Clifford+T and Clifford+CS	no proofs; the authors' coset-action data for Clifford+T
Modules	1,509	795
Code lines	376,627	338,866
Verification time		
Decided equalities	a Boolean reduced to `true`	`decide +kernel`
Check time, sixteen rows	2,399 s	9,305 s
Faster for	thirteen rows	three rows
Execution time		
Normal forms	executable functions	existence theorems or `noncomputable` definitions (460 declared)
Benchmarked procedure	matrix of each circuit, by products of stored matrices	simulation of each circuit on every basis vector
Compiled through	GHC (Haskell)	C
Per decision, Clifford, 100 gates	5.0 ms on 2 qubits, 24 s on 6	5.8 ms on 2 qubits, 0.61 s on 6

Table 2. The Agda and Lean formalizations compared. Counts are over the files under version control in the sixteen development folders of Table 1, without the shared libraries, and code lines exclude blank lines and comments. Check times are those of Table 1, and execution times those of Figure 7(c).

derives isomorphisms out of the quotient. In both, the circuit infrastructure, the structural rules, and general completeness arguments are shared between developments.

The semantic targets also differ. Most Agda developments work directly with exact representations of matrix entries. Lean states its matrix groups over Mathlib's real or complex numbers and proves bridges to them from computable exact rings. In particular, the Agda qudit Clifford development presents the constructed model of Appendix D, whereas Lean proves soundness and completeness against complex matrices for every odd prime. A common source therefore does not imply identical statements or intermediate lemmas.

Proof Development. Excluding blank lines and comments, the Agda formalization of Table 1 has about 377,000 lines and the Lean formalization about 339,000 (Table 2). Figure 7(a) compares them row by row, except that for Toffoli–Hadamard it shows another pair, one proof in both assistants (see below). Lean's is shorter in thirteen of the sixteen rows, by a factor of five or more for qudit

Clifford and qutrit Clifford, where the two take different routes. It is longer for Hadamard-Π, three-qubit Clifford+CS, and real Clifford+CH. Both counts include the derivations and certificates that scripts generate. The Agda proofs are explicit terms, built by pattern matching and equational reasoning, with reflection-based solvers for associativity and ring identities. Its library develops the group theory the proofs need, in about 13,200 lines against 1,200 in Lean. The Lean proofs rely on Mathlib's algebra and on tactics, with about 69,000 calls to automation such as `simp`, `omega`, `decide`, and `ring`.

Both implementations reuse normal-form, exact-synthesis, and translation arguments. Both also check generated derivations with a replay procedure that is proved correct. The proof assistant checks the computation and the theorem that a successful replay gives a derivation, so the scripts that search for derivations need not be trusted. For Clifford+CS, Lean constructs its own coset certificates. The trust checks reflect the different foundations. Every Agda module is `--safe` and uses no postulates, so the Agda corpus is constructive. The Lean corpus uses no `native_decide`, and its main theorems depend on no unfinished proof. An audit of 105 declarations reports only the axioms `propext`, `Classical.choice`, and `Quot.sound`.

Verification Time. Table 1 and Figure 7(b) give the check times, both measured on the same machine. In total Agda took 2,399 seconds and Lean 9,305, or 1,979 and 3,826 without real Clifford+CH, whose Lean build alone took 5,479 seconds, more than half of the total. Lean took between 1.2 and 4.8 times as long as Agda on twelve rows, and 13 times as long on real Clifford+CH. Its five-wire certificate modules `Family267` and `Family268` alone took about 1,741 and 1,369 seconds. Lean was faster for three rows, Clément's presentation, qudit Clifford, and qutrit Clifford, whose Lean proofs take different routes. Both assistants decide equalities of matrices and certificates by evaluation in the checker. Agda reduces a Boolean to `true`, and Lean uses `decide +kernel`.

Both columns were measured under WSL on an Intel Core i5-13400TEF with sixteen logical CPUs, about 26 GiB of RAM, and 64 GiB of swap. The Agda column comes from a check from scratch on 8 November 2026, with Agda 2.8.0 and version 2.4 of its standard library. We removed the repository's interface files, kept those of the standard library and EucDomain, and checked the shared library and then each row in an Agda process of its own, through a module that imports every module of the row. Agda's profiler reports the time spent on each module without its imports, and a row's time is the sum over its own modules, wherever they were checked. It leaves out process startup and the loading of interface files: the whole check, shared library included, took 3,210 seconds of wall time, and no process used more than 5.1 GiB. No other proof assistant ran during the check. The Lean column comes from a clean build on 5–6 November 2026, with Lean 4.30.0 and the pinned Mathlib revision. We removed QCert's build artifacts, kept the Mathlib and other dependency caches, and built the folders one at a time in dependency order with `LEAN_NUM_THREADS=2`. Each time is the elapsed wall time of a folder, including Lake startup and artifact generation, and a shared dependency is counted only in the folder that defines it. The two columns therefore measure different things, per-module checking time in Agda and folder build time in Lean. Proof constructions, module boundaries, parallelism, and artifact generation also differ, so the differences cannot be attributed to the proof assistants alone. We have yet to separate the time Lean spends running automation from the time it spends checking the resulting proofs.

The build covered all 810 tracked library modules, excluding three generator templates, and took **9,415.37 seconds (2 h 36 min 55 s)** in timed build commands. Of these modules, 808 passed. Two unused legacy helpers in real Clifford+CH failed, `ItemBFun` at `omega` proofs and `ItemASteps` at the heartbeat limit. Neither is imported by the main theorem chain or the audit, both of which passed, and the real Clifford+CH entry of Table 1 includes the whole attempted build. Beyond the sixteen development folders, the shared `Circuit`, `Exact`, `Matrix`, and `NumberTheory` folders

took 17.91, 15.51, 6.88, and 15.29 seconds, and the five root modules, including the audit, took 54.95 seconds. Lean builds of another project ran on the same machine during part of the run. We therefore re-timed the twenty folders and root modules they overlapped and report the second timing, during which the other project used about a quarter of one CPU on average. The two runs total 9,604 and 9,415 seconds, and the two timings of each development folder agree within 15%, except for Toffoli–Hadamard (178 and 135 seconds).

One Proof in Both Assistants. For Toffoli–Hadamard, the formalizations of Table 1 have 28,629 lines in Agda and 4,401 in Lean, and they prove the result by different routes. The Agda proof replays the derivations in the authors' own files as generated certificates, which make up 22,572 of its lines. The Lean proof was written from the paper, over an alphabet of macro gates, with a Knuth–Bendix system for the symmetric group S_8 that the kernel checks. To compare the two assistants on one proof, Figure 7(a) shows another pair instead (∗). We wrote a second Agda proof of the result, which finds its circuit derivations by a breadth-first search over the relations of Figure 4 of [3], and a coding agent translated it to Lean module for module, keeping its definitions, derivations, and proof route; this time we told the agent to follow the Agda. A coding agent also began translating the first Agda proof in the same way, porting its scripts so that they write the certificates in Lean, and we stopped it after 46 of the 203 modules: the relations, the 218 derivations replayed from the authors' files, the 642 certificates of the permutation fragment, and the frames built on them, 17,862 of the Agda proof's lines. On one proof the two assistants need about the same amount of text: 182,301 non-blank characters outside comments in Agda and 191,589 in Lean for the second proof and its translation, and 1,056,950 and 1,030,864 for the translated modules of the first. In lines the translation of the second proof is shorter, 5,672 against 7,161, only because its generated derivations put several rewrite steps on a line where Agda's put one, while the translated modules of the first, whose certificates take a step per line in both, are 6% longer, 18,946 against 17,862. The hand-written part of the second proof's translation is the larger of its pair, mostly because it proves the facts about S_8 that the Agda proof imports from the shared library. Lean took 164 seconds to check it and Agda 48, 3.4 times as long, while on the two proofs of Table 1, timed in the same way, Lean took 118 seconds and Agda 111, 1.1 times as long (Table 1, timing Lean's whole build, has 135 and 111). So the pair that Table 1 and Figure 7(b) report reflects the difference between the proofs more than the difference between the assistants.

The new result of Section 6 gives a third pair. Coding agents translated its 40 Agda modules to Lean module for module, again told to follow the Agda, and ported the scripts that generate its derivations, tables, and certificates so that they write Lean. Seven further Lean modules translate what the Agda imports from the shared library: the Coxeter presentation of the symmetric groups, signs and the alternating groups, the swap rules, and the rewriting engine. Again the two need about the same amount of text: 421,726 non-blank characters outside comments in Agda and 467,667 in Lean, or 431,042 without the seven library modules, in 13,135 and 12,177 lines. Lean took 277 seconds to check the translation and Agda 85, 3.3 times as long, close to the ratio of the Toffoli–Hadamard pair. Agda checks the 496-step derivation of a base commutation by one evaluation, whereas one kernel check of it in Lean had not finished after eight minutes, at 11 GB of memory, so the translation cuts each long derivation into segments of at most 40 steps. These six times were measured on 10 November 2026 on the machine of Table 1, by its method, with no other proof assistant running.

Execution Time. The Agda normal forms are executable functions. The Lean developments obtain normal forms either as existence theorems or as `noncomputable` definitions. For example, the Lean proofs for the Clifford and real stabilizer circuits of [49, 60] show that every circuit equals a

normal form without defining a function that computes it, and the exact synthesis of the two-level groups is noncomputable. In all, 460 definitions in the sixteen Lean developments are declared `noncomputable`, so there is no Lean normalizer to time. Both formalizations can, however, execute the procedure that decides whether two circuits have the same matrix, which their soundness proofs run. We compiled this procedure for the Clifford circuits of [60] to native code, through Agda's GHC backend and Lean's C backend. We then timed both on an Intel Core i5-14400, on the same random circuits of 100 gates over H, S, and CZ, deciding that each circuit w has the matrix of $w\,S^4$ (Figure 7(c)). On two qubits Agda is slightly faster, at 5.0 against 5.8 milliseconds per decision. From three qubits on Lean is faster, by a factor of 2.1 on three qubits and 40 on six, where Agda takes 24 seconds and Lean 0.61. The gap comes mostly from the procedures rather than the compilers. Agda computes each circuit's matrix as a product of stored $2^n \times 2^n$ matrices, so its work grows about eightfold with each qubit. Lean simulates the circuit on each basis vector, so its work grows about fourfold.

D Case Study: Multi-Qudit Clifford Completeness

The multi-qudit Clifford result of Bian, Li et al. [12] shows how QCert turns a circuit completeness proof into reusable, checked components. Here each qudit has odd prime dimension p; the result covers every number n of qudits and every such p. Its sixteen rules must derive every valid equation between Clifford circuits. It illustrates two design choices of Section 3.2, defining the semantic group independently of the circuit syntax and assembling completeness proofs from general theorems about groups, as Figure 5 summarizes.

D.1 From Circuit Equations to Checked Derivations

The circuits use the Fourier gate H, the phase gate S, and the controlled-phase gate CZ. We first work projectively, identifying operators that differ only by a global phase. Figure 11 gives a small example of the translation into QCert: a swap is a word in the generators, its laws are circuit equations, and a derived equality records the rule used at each step. The framework supplies the structural rules that let the same derivation apply inside a larger circuit. The corresponding Agda definitions and proof terms are in Appendix D.4.

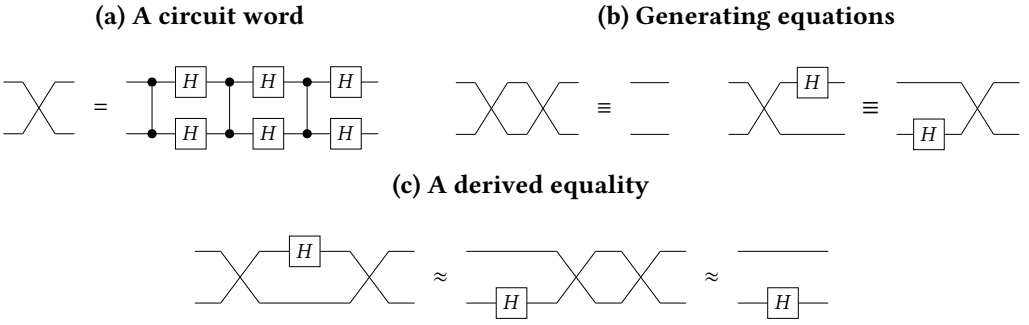


Fig. 11. Circuit syntax, rules, and reasoning in the projective Clifford formalization. Crossed wires denote swap, and connected dots denote CZ. Diagrams are read from right to left. In (c), moving H across one swap and cancelling the two swaps moves H to the other wire. The generating equations (≡) justify the derived equivalence (≈).

To prove completeness, we must also specify what a circuit means. We first consider its action on Pauli operators, ignoring phase factors. Each wire contributes a pair (a, b) modulo p, describing

$X^a Z^b$ for the generalized Pauli shift and phase gates X and Z. The induced transformations form the symplectic group $\mathrm{Sp}(2n, \mathbb{Z}_p)$, which preserves Pauli commutation. This first stage identifies Clifford operators that differ by a Pauli; the next stage restores that information. This group is defined using linear algebra, independently of circuits and their rewrite rules. Completeness at this stage therefore requires a proof that equal transformations give derivable equations in the symplectic theory.

D.2 Normalize the Factors, Then Compose

The first step is a normal-form proof for this symplectic group. Figure 12 shows the recursive shape: a canonical block accounts for one wire, and normalization continues on the remaining wires. The formalization proves that every circuit rewrites to this shape, that each rewrite preserves its meaning, and that the normal form is uniquely determined by that meaning. Consequently, circuits denoting the same transformation rewrite to the same normal form and are equivalent. This is the normalization strategy of Section 2, instantiated with the source paper's circuit blocks; the datatypes and normalization table are in Appendix D.5.

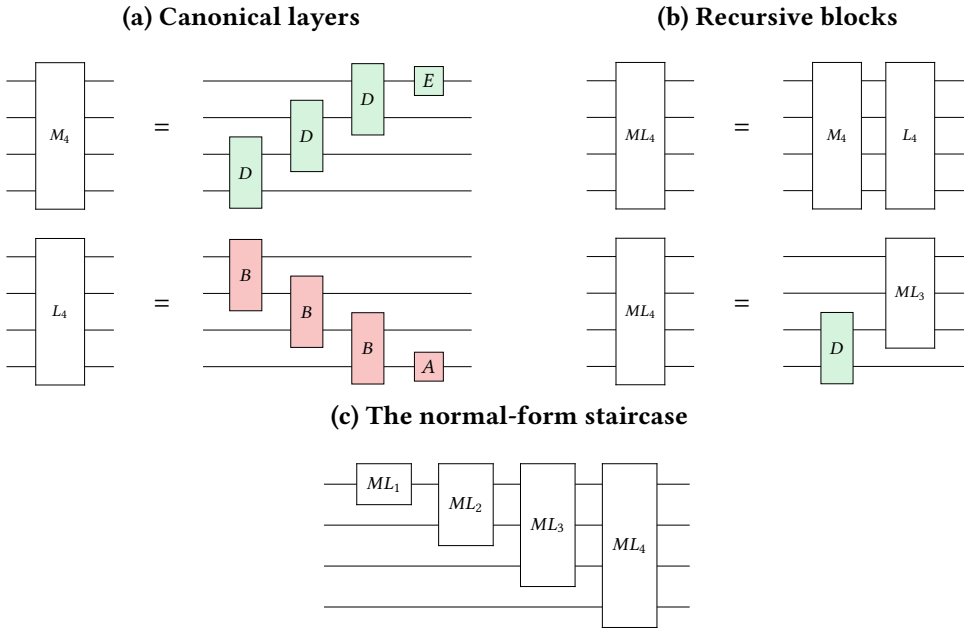


Fig. 12. The symplectic normal form, illustrated on four wires. (a) The M and L layers use the source paper's parameterized circuit blocks A, B, D, and E. (b) An ML block combines the two layers or extends a smaller ML block by a D block. (c) One block per width gives the staircase. Its uniqueness is the key completeness obligation; Paulis and scalar phases are restored separately.

Symplectic transformations retain only the Clifford action modulo Paulis. To restore the Paulis, we present their phase space $V = \mathbb{Z}_p^{2n}$ separately and combine the two presentations into the independently defined model $V \rtimes \mathrm{Sp}(2n, \mathbb{Z}_p)$. The semidirect product records how a symplectic transformation acts on Pauli exponents. QCert's general composition theorem asks us to prove that this action respects the relations of both factors. Once these obligations are checked, completeness of the combined presentation follows from completeness of the factors.

This replaces the most difficult part of following the source proof directly. Its passage from symplectic to projective completeness tracks a Pauli correction through each rewrite. We instead discharge the compatibility obligations once and use the framework's semidirect-product theorem. Verified translations then recover the paper's generators and relations. Finally, the scalar-extension theorem restores global phases, with the necessary phase corrections checked explicitly (Figure 5). The construction-specific calculations remain in the case study; the composition theorems can be reused by other developments.

D.3 What the Formalization Establishes

Agda checks the presentations of the symplectic, projective, and scalar-extension models for every odd prime p and every width n, with no postulates or unfinished proofs. In particular, it checks both normalization and the translations back to the source rule sets. This demonstrates how QCert can support a proof that needs substantial mathematical restructuring while keeping each obligation explicit.

The semantic boundary is also explicit. The Agda development presents independently constructed algebraic models; identifying them with the unitary Clifford groups and matching the paper's gate phases uses external mathematics. The Lean development proves soundness and completeness directly against complex matrices (Appendix C). The rest of this appendix gives the details. We first fix the gate conventions and state the formal theorems (Appendix D.4). We then prove symplectic completeness by normalization (Appendix D.5), restore Paulis and recover the source rule sets (Appendix D.6), and restore scalar phases (Appendix D.7). Appendix D.8 records the implementation and the boundary between the Agda proof and the external identification with unitary groups. Thus the case study shows both how the proof is assembled and exactly which connection each proof assistant checks.

D.4 Gate Conventions, Syntax, and Theorem Statements

We use the gate conventions of Bian, Li et al. [12]. Fix an odd prime p and let $\omega = e^{2\pi i/p}$. On the basis $|0\rangle, \ldots, |p-1\rangle$ indexed by $\mathbb{Z}_p$, the gates $X|j\rangle = |j+1\rangle$ and $Z|j\rangle = \omega^j|j\rangle$ generate, with the phases ω^t, the Pauli group $\mathcal{P}_n$ on n qudits. The gates

$$H\colon |j\rangle \mapsto \frac{\delta_p}{\sqrt{p}} \sum_k \omega^{jk}|k\rangle, \qquad S\colon |j\rangle \mapsto \omega^{j(j-1)/2}|j\rangle, \qquad CZ\colon |j\rangle|l\rangle \mapsto \omega^{jl}|j\rangle|l\rangle$$

generate, with the same phases, the Clifford group C_n. Here $\delta_p = (-i)^{(p-1)/2}$, which makes the determinant of H equal to 1. Modulo scalars, we write $\overline{C}_n = C_n/\langle\omega\rangle$ and $\overline{\mathcal{P}}_n = \mathcal{P}_n/\langle\omega\rangle \cong \mathbb{Z}_p^{2n}$. Then $C_n/\mathcal{P}_n \cong \overline{C}_n/\overline{\mathcal{P}}_n \cong \mathrm{Sp}(2n, \mathbb{Z}_p)$ and $\overline{C}_n \cong \overline{\mathcal{P}}_n \rtimes \mathrm{Sp}(2n, \mathbb{Z}_p)$. We do not formalize these last two isomorphisms, because our semantic groups are *defined* as their right-hand sides. Bian, Li et al. [12] also include -1 among the global phases, so that the swap and the multiplier gates belong to C_n. We work mainly in the projective quotient, where -1 makes no difference and the relations are slightly simpler.

Using the framework of Section 3, we encode circuits, rules, and reasoning with the rules. Figure 11 shows the diagrams for the following encodings. We draw circuits in matrix-multiplication order, the reverse of the usual circuit order, so $w \bullet v$ applies v first but draws it on the right. For example, this is the swap:

```
Ex : Word (Gen (₂₊ n))
Ex = CZ • H ↓ • H ↑ • CZ • H ↓ • H ↑ • CZ • H ↓ • H ↑
```

A rule set is an inductive family indexed by the width, with one constructor per rule. Each rule is stated once, on the bottom wires of every width at which it fits. For example, these are two of the

fifteen constructors of our primary rule set _SRel,_===_, the order of the swap and the rule moving H across it:

```
data _SRel,_===_ : (n : ℕ) → WRel (Gen n) where
  order-Ex     : (₂₊ n) SRel, Ex ^ 2 === ε
  semi-Ex-H↑ : (₂₊ n) SRel, Ex • H ↑ === H ↓ • Ex
```

Reasoning with the rules is equational reasoning in the congruence ≈ they generate, with combinators for symmetry, associativity, and rewriting the left or right factor of a product. For example, conjugating by the swap moves H down a wire:

```
conj-Ex-H↑ : Ex • H ↑ • Ex ≈ H ↓
conj-Ex-H↑ = begin
  Ex • H ↑ • Ex   ≈⟨ sym assoc ⟩
  (Ex • H ↑) • Ex ≈⟨ cleft axiom semi-Ex-H↑ ⟩
  (H ↓ • Ex) • Ex ≈⟨ assoc ⟩
  H ↓ • Ex • Ex   ≈⟨ cright axiom order-Ex ⟩
  H ↓ • ε         ≈⟨ right-unit ⟩
  H ↓ ■
```

Theorem D.1 (Paper-V1.presentation). *For every width n, the fifteen rules of* `Paper-V1` *(Section D.6) present the projective Clifford model:*

```
presentation : ∀ n → (PapV1.Clifford-Relations._QRel,_===_ n) IsPresentationOf (Pauli⋊Sp n)
```

These rules are the relations of Bian, Li et al. [12] read modulo scalars, with its rule (C4) in an equivalent form. The relation _QRel,_===_ closes _SRel,_===_ under the structural rules (Section 3), and Pauli⋊Sp n is the semidirect product of $\mathbb{Z}_p^{2n}$ by $\mathrm{Sp}(2n, \mathbb{Z}_p)$ (Section D.6). The record _IsPresentationOf_ (Section 3) gives a group isomorphism from circuits modulo ≈ to $\overline{\mathcal{P}}_n \rtimes \mathrm{Sp}(2n, \mathbb{Z}_p)$. That this map is well defined is *soundness*, since each rule holds. That it is injective is *completeness*, and that it is surjective says that every element of the model is represented by a circuit.

We refine the normal form of Bian, Li et al. [12] for symplectic operators, which are Clifford operators modulo Paulis, into a doubly inductive definition (Section D.5), and we prove that normal forms are unique:

Theorem D.2 (unique-nf). *Normal forms with equal symplectic denotations are equal:*

```
unique-nf : ∀ n {u v : NF n} → [[ ⌈ u ⌉ ]] ≈ˢ [[ ⌈ v ⌉ ]] → u ≡ v
```

The Route. We mostly follow the paper. Its normal boxes, its symplectic normal form, and its parametrized *box relations* reappear below. A box relation says that pushing a gate through a normal box yields an updated box and residual *dirty* gates. We also keep the reduction to eighteen relations of its Theorem 4.4, a step the paper already delegated to Agda. The one major divergence is the step that promotes symplectic to projective completeness, its Proposition 4.9. There, the paper corrects each rule $C_l = C_r$ to $C_l = C_r\,;P$ for the unique Pauli P that makes it projectively sound. It then replays the symplectic rewrites, pushing each correction to the end, where it becomes "a potentially different Pauli". We did not see how to verify this argument at reasonable cost. Instead, we work in a syntax where Paulis are trivial, since symplectic circuits have only H, S, and CZ and denote $\mathrm{Sp}(2n, \mathbb{Z}_p)$ itself. We then reintroduce Paulis and scalars by construction (Figure 13). The Pauli bookkeeping of Proposition 4.9 is thus done once, by a theorem about semidirect products that the framework proves for arbitrary groups.

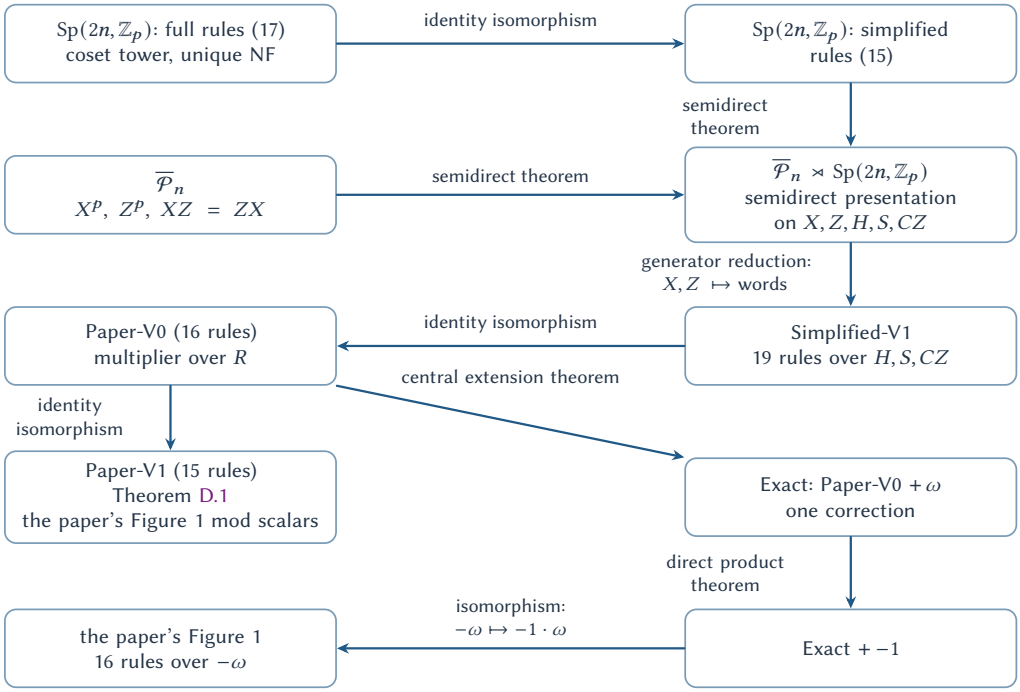


Fig. 13. The chain of presentations used to formalize the qudit Clifford result of Bian, Li et al. [12]. Every arrow is a verified presentation theorem or isomorphism of presentations. The symplectic stages present $\mathrm{Sp}(2n, \mathbb{Z}_p)$, and the projective chain from the semidirect presentation to Paper-V1 presents $\overline{\mathcal{P}}_n \rtimes \mathrm{Sp}(2n, \mathbb{Z}_p)$. "Exact" is a central extension by $\langle \omega \rangle$, and the final two stages also include the direct factor $\langle -1 \rangle$.

D.5 Normal Forms for the Symplectic Factor

We now give the datatypes and proof obligations behind Figure 12. We model $\mathrm{Sp}(2n, \mathbb{Z}_p)$ as the group of $\mathbb{Z}_p$-linear bijections of the phase space Pauli n = Vec $(\mathbb{Z}_p \times \mathbb{Z}_p)$ n that preserve the symplectic form. An element of the phase space lists, for each wire, the exponents of $X^a Z^b$ up to phase. Against this semantics, completeness rests on the normal form, which we refine into a doubly inductive datatype:

```
M L ML : ℕ → Set
M 0 = ⊤
M (1+ n) = Vec D n × E
L 0 = ⊤
L (1+ n) = Vec B n × A
ML 0 = ⊤
ML 1 = M 1 × L 1
ML m@(2+ n) = M m × L m
            ⊎ D × ML (1+ n)
```

An M-row is a column of D boxes ending in an E box, the paper's X-normal layer. An L-row is a column of B boxes ending in an A box, its Z-normal layer, with the A box at the bottom. An ML box at width $m \geq 2$ is either a full pair of rows or a bottom D box followed by an ML box one wire

up, so it is a spine of D boxes capped by a pair of rows. This is the first induction, on the width of a single coset representative. The second induction is the staircase:

```
NF : ℕ → Set
NF 0 = ⊤
NF (1+ n) = NF n × ML (1+ n)

[_] : ∀ {n} → NF n → Word (Gen n)
[_] {0}     tt       = ε
[_] {1+ n} (nf , ml) = [ nf ] ↑ • [ ml ]ᵐˡ
```

The Table. The coset table is the paper's "push a gate through a normal box", written as a structurally recursive function ract with its soundness law ract-sound (about 5,600 lines together):

```
ract       : ML (1+ n) → Gen (1+ n) → Circuit n × ML (1+ n)
ract-sound : ∀ (c : ML (1+ n)) g → let (b' , c') = ract c g in [ c ]ᶜ • [ g ]ʷ ≈ b' ↑ • [ c' ]ᶜ
```

Each clause of ract-sound is derived from the fifteen simplified symplectic relations. For instance, pushing S through an E box is one step on powers of S, namely $[\,b\,]^{\mathrm{e}} \bullet S = S^{-b} \bullet S \approx S^{-(b-1)} = [\,b-1\,]^{\mathrm{e}}$. A circuit is rewritten to its normal form by repeating this step gate by gate, so every rewrite is traced to a relation.

Uniqueness. The symplectic group acts on Vec Pauli$_1$ n, a $2n$-dimensional symplectic $\mathbb{Z}_p$-vector space in which Pauli$_1$ is the two-dimensional symplectic subspace of a single wire. The elements that fix the bottom wire's subspace pointwise form a copy of $\mathrm{Sp}(2(n-1),\mathbb{Z}_p)$ acting above the bottom wire, and the *ML* boxes represent its cosets. Every $f \in \mathrm{Sp}(2n,\mathbb{Z}_p)$ factors as $f = f' \circ c_n$, with $c_n \in ML_n$ and f' acting above the bottom wire, and c_n is unique because the image of the bottom wire determines the box. Recursing on f' writes f uniquely as $c_1 \circ c_2 \circ \cdots \circ c_n$ with $c_k \in ML_k$, which is Theorem D.2.

Surjectivity. Every symplectic transformation is the action of some circuit. We prove this constructively by clearing the bottom wire and recursing on the rest. This is the paper's Lemma 3.7, with the coset box computed rather than asserted:

```
Theorem-LM : ∀ {n} (p q : Pauli n) → sform p q ≡ ₁ → ∃ λ (lm : ML n) → act [ lm ]ᵐˡ p ≡ pZ₀ × act [ lm ]ᵐˡ q ≡ pX₀
```

Completeness. Circuits with the same denotation rewrite through the table to their normal forms. These normal forms denote the same map by soundness, so they are equal by Theorem D.2, and the circuits are equivalent. This argument is the framework's by-normalization (Figure 3a). With surjectivity, it shows that the simplified relations, which their own module also calls _SRel,_===_, present the symplectic group:

```
presentation : ∀ n → (n SRel,_===_) IsPresentationOf (Sp-group n)
```

D.6 Restoring Paulis and the Source Rule Sets

$\overline{\mathcal{P}}_n \cong (\mathbb{Z}_p \times \mathbb{Z}_p)^n$ is presented over X and Z on each wire by $X^p = Z^p = \varepsilon$ and $ZX = XZ$. Its semantic group is the phase space that $\mathrm{Sp}(2n,\mathbb{Z}_p)$ acts on, and its normal form is $X^a Z^b$ on each wire. Combining this with the symplectic presentation of Section D.5 is an instance of the framework's theorem for semidirect products. The resulting presentation has the relations of both factors together with the conjugation relations $h\,n\,h^{-1} = \mathit{conj}\,h\,n$, where the action conj spells out the paper's Lemmas 2.24–2.25. The theorem asks that the action respect the relations of each factor, and we prove this semantically. The conjugates of congruent words have equal denotations by soundness, so they are congruent by Pauli *completeness*. For symplectic rules this is respects-Δ, and

it corresponds to the paper's "Pauli complete" rules (its Definition 4.7). Since every premise is a theorem, the semidirect presentation of Pauli⋊Sp is unconditional.

Reaching the Paper's Rules. The semidirect presentation has generators X, Z, H, S, and CZ and $3 + 15 + 14$ relations (Pauli, symplectic, and conjugation), whereas the paper's rule sets have only H, S, and CZ. Three isomorphisms of presentations close this gap (Figure 13), through `Simplified-V1` (nineteen rules), `Paper-V0` (sixteen), and `Paper-V1` (the fifteen of Theorem D.1). Along the way, X and Z become words, and the Pauli and conjugation rules become theorems.

The first isomorphism, from the semidirect presentation to `Simplified-V1`, is between different alphabets. Its map f sends S to $R = S \bullet Z^{1/2}$, and its inverse h sends S to $Z^{-1/2} \bullet S$. In particular, f induces the embedding of the symplectic group into the projective Clifford group. The nineteen rules of `Simplified-V1` are the fifteen simplified symplectic rules together with $(SH)^3 = \varepsilon$, $H^2SH^2S = SH^2SH^2$, and the two rules pushing X through CZ. From these, f-well-defined derives the images under f of the three Pauli rules and the fourteen conjugation rules.

The second isomorphism, from `Simplified-V1` to `Paper-V0`, is the identity on circuits. The two rule sets share six rules word for word and three more up to the spelling of the multiplier, and each derives the rest of the other. The larger direction is `Paper-V0`'s module `Lemmas`, of about 4,750 lines.

The third isomorphism, from `Paper-V0` to `Paper-V1`, changes how the multiplier $M_x \colon |j\rangle \mapsto |xj\rangle$ is spelled as a word. `Paper-V0` spells it over R, as $M_x = R^{x^{-1}}HR^xHR^{x^{-1}}H$. The paper's (T1) spells it over S, up to a phase, as $M_x =$ SHS' $x\ x^{-1}$, where SHS' $a\ b = Z^{(b-1)/2}X^{(1-a)/2}S^bHS^aHS^bH$. Over S, the rule semi-MR reads $M_gS^{g^2}Z^{(g^2-g)/2} = SM_g$. `Paper-V1` restates the four multiplier rules over SHS' and drops one rule. Since M_1 is now the word $(SH)^3$, M-power at $k = 0$ already gives $(SH)^3 \approx \varepsilon$. Over R, M_1 is the different word $(RH)^3$, so that axiom had to stay. The bridge $R^aHR^bHR^aH \approx$ SHS' $b\ a$ for $ab = 1$ (`Shared.PauliBase`) carries the multiplier rules across, which gives Theorem D.1. The paper's own rule (C4) states the conjugation of S by M_g in the other direction. Each of (C4) and semi-MR derives the other (SemiMS), so Figure 1 of Bian, Li et al. [12] read modulo scalars is isomorphic to `Paper-V1`.

D.7 Restoring Scalar Phases

The theorem extension-presentation extends a projective presentation by a scalar. We apply it to `Paper-V0`, which has order-SH as an axiom. We add ω with $\omega^p = \varepsilon$, let ω commute with every gate, and *correct* each rule by its scalar. The correction is ε except on order-SH, the one rule that does not hold exactly, since $(SH)^3 = \omega^{-1/8}$. The exponent $-1/8$ is computed in $\mathbb{Z}_p$, where it equals $(p^2-1)/8$. The multiplier words now denote $|j\rangle \mapsto |xj\rangle$ only up to a scalar.

The semantic extension is the Weyl cocycle on the natural semidirect product, $c((P,A),(Q,B)) = \frac{1}{2}\,\mathrm{sform}(P,\ A \cdot Q)$. Here $A \cdot Q$ is the symplectic part acting on the Pauli vector, and sform is the symplectic form of the phase space, the exponent in $PQ = \omega^{\mathrm{sform}(P,Q)}\,QP$. The cocycle identity follows from the bilinearity and the symplectic invariance of sform. The cocycle of the presented group is the pullback of c along the presentation by `Paper-V0`, so the two agree by construction. The identification with unitary operators uses Weyl multiplication [37, Eq. (4), Theorem 3] and a *linear* Weil representation with exact covariance [38, Theorem 1.3.2]. The operators $U(e,v,A) = \omega^e W(v)\rho(A)$ satisfy

$$U(e,v,A)\,U(f,w,B) = U\big(e + f + \tfrac{1}{2}\mathrm{sform}(v,Aw),\ v + Aw,\ AB\big).$$

This is the Heisenberg–Weil representation [39, §1.3]. Conjugation on Weyl operators determines A and v, so the representation is faithful and its scalars are exactly $\langle \omega I\rangle$ (at width zero, $e \mapsto \omega^e$).

Adjoining $-I$ gives its direct product with $\mathbb{Z}_2$, whose scalars are $\langle -\omega I \rangle$. These finite phase choices differ from the full unitary normalizer, which contains $U(1)$. With no hypotheses, we obtain:

```
presentation-exact : ∀ n → (n Exact._===_) IsPresentationOf (PE.Presented-group n (exact-generator-data n))
```

Agda also shows that the trivial cocycle cannot realize the correction of order-SH (split-forces).

The Rules as Printed. Figure 1 of Bian, Li et al. [12] is stated over $-\omega$. Its multipliers carry Legendre symbols and its swap carries the sign $(-1)^{(p-1)/2}$, so it needs the -1 that the exact group lacks, since the scalars of that group have odd order. Adjoining -1 as a central involution presents the exact group times $\mathbb{Z}_2$, which is Clifford±. Figure 1 is isomorphic to this rule set by $-\omega \mapsto -1 \cdot \omega$, where the scalar of each exact rule is determined by reading its lift back. This is the paper's Theorem 4.10, stated against our group:

```
presentation-Figure1 : ∀ n → (n F._===_) IsPresentationOf (Clifford± n)
```

D.8 Implementation and Semantic Boundary

This development is checked by Agda 2.8 with standard library 2.4 under `--safe`, with no postulates and no holes. The development has 215 modules and about 58,000 lines of code, of which about 27,000 are for the symplectic group, 18,000 for the projective chain, and 13,000 for the scalars. These counts exclude the shared library (Appendix C).

The Agda development does not formalize the two isomorphisms of Section D.4 [37, 38], and no matrix appears in it. The representation argument above is external mathematics, and matching the paper's gate phases remains a convention check [12, 52]. The Lean development needs neither, since it proves soundness and completeness against complex matrices directly (Appendix C).

We used versions of Claude for parts of this formalization: composing box relations to prove ract-sound, defining the symplectic group, and establishing the equivalence between the code's relations and those in Figure 1 of Bian, Li et al. [12], read modulo scalars. Claude also wrote presentation-exact and the layer for Figure 1 with -1.

E Proofs for Four-Bit Reversible Circuits

This appendix proves Theorems 6.1 and 6.2 as the Agda development [4] does. Each subsection names the modules of `Examples/Groups/XCXCCX-4bit` that it follows, and each statement names its Agda counterpart.

E.1 Conventions and Outline

The syntax and the proof engine are the modules Syntactics and Kit. Wires are numbered from 0 at the bottom, so on four wires the top wire is wire 3 and the *lower wires* are 0, 1, and 2. A state of k wires is a bit string $b_{k-1} \cdots b_0$, the bit of the top wire first. Circuits are words in diagram order, and products of permutations follow the same order, so uw applies u and then w. We write $u \approx w$ when u and w are equal modulo $\mathcal{T}$, and ε for the empty circuit. On four wires, the halves H_0 and H_1 are the sets of states in which wire 3 is 0 and 1, and a permutation that preserves them acts on H_0 and on H_1 as two permutations of the eight states of the lower wires, its blocks.

The gate X acts on wire 0, CX has its control on wire 1 and its target on wire 0, CCX has its controls on wires 2 and 1 and its target on wire 0, and the structural swap σ exchanges the wires 0 and 1. A gate in any other position is one of these, shifted up or between swaps, and an open control is a filled one between two X gates on its wire. We write X_t, $\mathrm{CX}_{c \to t}$, and $\mathrm{CCX}_{ab \to t}$ for the gates with target t and controls c, or a and b in either order, and σ_d for the swap of the wires d and

$d + 1$. In this notation the equations of Figure 8 are

(T1) $\mathrm{X}_0\,\mathrm{X}_0 = \varepsilon$, (T2) $\mathrm{CX}_{1\to0}\,\mathrm{X}_1\,\mathrm{CX}_{1\to0}\,\mathrm{X}_1 = \mathrm{X}_0$,

(T3) $\mathrm{CX}_{1\to0}\,\mathrm{CX}_{0\to1}\,\mathrm{CX}_{1\to0} = \sigma_0$, (T4) $\mathrm{CCX}_{21\to0}\,\mathrm{X}_2\,\mathrm{CCX}_{21\to0}\,\mathrm{X}_2 = \mathrm{CX}_{1\to0}$,

(T5) $\sigma_1\,\mathrm{CCX}_{21\to0}\,\sigma_1 = \mathrm{CCX}_{21\to0}$,

(T6) $\mathrm{CX}_{0\to1}\,\mathrm{CCX}_{21\to0}\,\mathrm{CX}_{0\to1} = \mathrm{CCX}_{20\to1}\,\mathrm{CCX}_{21\to0}\,\mathrm{CCX}_{20\to1}$,

(T7) $\mathrm{CCX}_{32\to1}\,\mathrm{CX}_{1\to0}\,\mathrm{CCX}_{32\to1} = \mathrm{CCX}_{32\to0}\,\mathrm{CX}_{1\to0}$.

Derivations are checked by a small proof engine. An equation is a pair of lists of generators with a derivation between their circuits, and it holds on the bottom wires of a circuit of any width. A new equation is derived by a list of steps, each of which reorders generators on disjoint wires or rewrites an occurrence of one side of a known equation into the other, and Agda checks the derivation by running the steps. Most derivations are generated by scripts that search for them. A fact is *decided by evaluation* when it is stated as a Boolean that Agda's type checker computes to be true.

The proof has five parts.

(1) On at most three wires, the transpositions of consecutive states along a Gray code are circuits that satisfy the Coxeter relations of the symmetric group, which gives completeness (Appendices E.2 and E.3).
(2) On four wires, every circuit is a product of gates placed on arbitrary wires, an equation on three of the four wires holds when it holds semantically, and a few four-wire equations are derived from (T7) (Appendix E.4).
(3) A presentation of the alternating group by the Reidemeister–Schreier method reduces completeness on four wires to the circuits whose gates never target wire 3 (Appendices E.5 and E.6).
(4) Such a circuit is a product of controlled matchings followed by a circuit on the lower wires, and two such circuits with the same function are equal modulo $\mathcal{T}$ (Appendices E.7–E.10).
(5) The groups and the block normal form follow from completeness (Appendices E.11 and E.12).

The normal form is thus a consequence of completeness, not the route to it. A proof by the first strategy of Figure 3, which absorbs one gate at a time into a normal form, would rest on finite certificates of about 380,000 deductions. The development uses presentations of permutation groups instead, so that what is checked by computation stays small.

E.2 Soundness and Involutions

This part follows the modules Semantics and Involutions.

Proposition E.1 (sound). *On any number of wires, if $u \approx w$ then $[\![u]\!] = [\![w]\!]$.*

Proof. The two sides of each of (T1)–(T7) denote the same function at the width at which the equation is drawn, which is decided by evaluation on all states of that width. The structural rules hold at every width, by computation on the bits of the wires that they touch. The claim follows by induction on the derivation. □

Lemma E.2 (Involutions). $\mathrm{CX}\,\mathrm{CX} \approx \varepsilon$ *and* $\mathrm{CCX}\,\mathrm{CCX} \approx \varepsilon$, *on the bottom wires of a circuit of any width.*

Proof. Gates have no formal inverses, so each derivation cancels a circuit only after it has derived an inverse for it. On two wires, let $a = \mathrm{CX}_{1\to0}$ and $b = \mathrm{CX}_{0\to1}$. Equations (T1)–(T3) give $baab \approx \varepsilon$, and then $aa \approx a(baab)a = (aba)(aba) \approx \sigma_0\sigma_0 \approx \varepsilon$ by (T3). On three wires, let

$c = \mathrm{CCX}_{21\to 0}$, $e = \mathrm{CCX}_{20\to 1} = \sigma_0 c\, \sigma_0$, $x = \mathrm{X}_2$, and $m = xex$. From (T1), (T3), (T4), (T6), and $\mathrm{CX}\,\mathrm{CX} \approx \varepsilon$, the development derives $mm \approx \varepsilon$, and then $cc \approx \sigma_0 ee\, \sigma_0 \approx \sigma_0 xmmx\, \sigma_0 \approx \varepsilon$. □

E.3 At Most Three Wires

This part follows the modules Coxeter, ByCoxeter, Width1, Narrow2, Width2, and Width3. The library of QCert proves the Coxeter presentation of $\mathrm{Sym}(N)$ [28]. Its generators are the transpositions s_d of d and $d+1$, for $0 \le d \le N-2$, and its relations are $s_d s_d = 1$, $s_d s_{d+1} s_d = s_{d+1} s_d s_{d+1}$, and $s_d s_e = s_e s_d$ for $e \ge d+2$. Two words in the s_d, called *Coxeter words*, with the same permutation are equal modulo these relations, and every permutation is that of a Coxeter word. The inverse p^{-1} of a Coxeter word p is the reversed word.

A *Coxeter system* for a set of circuits consists of a numbering of the objects that the circuits permute by $0, \ldots, N-1$ and of circuits $T_0, \ldots, T_{N-2}$ such that T_d acts on the objects as the transposition of the objects numbered d and $d+1$, and the T_d satisfy the Coxeter relations modulo $\mathcal{T}$. The circuit $\langle p \rangle$ of a Coxeter word p replaces each s_d by T_d.

Lemma E.3 (Coxeter.perm-complete, ByCoxeter). *In a Coxeter system, two Coxeter words with the same permutation have circuits that are equal modulo $\mathcal{T}$. If the objects are the 2^k states of k wires and every generator of the circuits on k wires is equal modulo $\mathcal{T}$ to the circuit of a Coxeter word, then $\mathcal{T}$ is complete on k wires.*

Proof. The two words are equal modulo the Coxeter relations, and each relation holds for the T_d. For the second claim, translating a circuit u generator by generator gives a Coxeter word p_u with $u \approx \langle p_u \rangle$. If $[\![u]\!] = [\![w]\!]$, then by Proposition E.1 the circuits $\langle p_u \rangle$ and $\langle p_w \rangle$ denote the same function, so p_u and p_w have the same permutation, and $u \approx \langle p_u \rangle \approx \langle p_w \rangle \approx w$. □

Lemma E.4 (frame lemma, Coxeter.frame≈). *In a Coxeter system, let b be a circuit that commutes modulo $\mathcal{T}$ with every T_i for $i \ge j$, and let* $\mathrm{Fr}(b, p) = \langle p \rangle\, b\, \langle p^{-1} \rangle$ *be its* frame *by a Coxeter word p. If $\langle p \rangle$ and $\langle p' \rangle$ carry the same objects to those numbered $0, \ldots, j-1$, then $\mathrm{Fr}(b, p) \approx \mathrm{Fr}(b, p')$.*

Proof. The permutation of the word $r = p'^{-1} p$ fixes the numbers below j, so it is the permutation of a Coxeter word r' in the s_i with $i \ge j$, by the surjectivity of the presentation of a smaller symmetric group. By Lemma E.3, $\langle r \rangle \approx \langle r' \rangle$, which commutes with b. Since $\langle p \rangle \approx \langle p' \rangle \langle r \rangle$, the two frames are equal. □

For every list of distinct objects there is a Coxeter word, the *frame word* of the list, whose circuit carries the objects of the list to those numbered $0, 1, \ldots,$ in order.

Proposition E.5 (complete$_1$, complete$_2$, complete$_3$). *The theory $\mathcal{T}$ is complete on one, two, and three wires.*

Proof. *One wire.* The only generator is X, and by (T1) a circuit is equal to X or to ε according to the parity of its length. Its function shows the parity on the state 0.

Two wires. Number the states 00, 01, 11, 10, and let $T_0 = \mathrm{X}_1\, \mathrm{CX}_{1\to 0}\, \mathrm{X}_1$, $T_1 = \mathrm{CX}_{0\to 1}$, and $T_2 = \mathrm{CX}_{1\to 0}$. They are involutions by Lemma E.2. By (T2), $T_0 T_2 \approx \mathrm{X}_0 \approx T_2 T_0$, and by (T3), $T_1 T_2 T_1 \approx \sigma_0 \approx T_2 T_1 T_2$. The gate X_1 commutes with T_1, the CNOT onto its wire, and conjugates T_2 to T_0, so it carries the braid relation of T_1 and T_2 to that of T_1 and T_0. The generators are $\mathrm{X}_0 \approx T_2 T_0$, $\sigma_0 \approx T_2 T_1 T_2$, $\mathrm{CX}_{1\to 0} = T_2$, and $\mathrm{X}_1 \approx T_2 T_0 T_1 T_2 T_0 T_1$, so Lemma E.3 applies.

Three wires. Number the states along the Gray code 000, 001, 011, 010, 110, 111, 101, 100, in which consecutive states differ in one bit. Two states that differ in one bit are the ends of an edge of the cube, and their *edge Toffoli* is the Toffoli gate onto the wire of that bit, with each control filled or open according to the bit that the two states share on its wire. There are twelve edge Toffolis, and T_d is

that of the states numbered d and $d+1$. For example, $T_4 = \mathrm{CCX}_{21\to 0}$ and $T_5 = \mathrm{CCX}_{20\to 1}$. The Coxeter relations are derived in three groups. First, take the four edge Toffolis onto wire 0, which are T_0, T_2, T_4, and T_6. By (T4) and (T5), the products T_4T_2 and T_4T_6, in either order, are the CNOTs from wire 1 and from wire 2 onto wire 0, so T_4 commutes with T_2 and with T_6. By (T2) on the wires 1, 0 and on the wires 2, 0, the gate X_0 is $T_4T_6T_2T_0$ and also $T_4T_2T_6T_0$, so T_2 commutes with T_6. The commutations with T_0 follow, and X_0 commutes with each of the four. The same holds for the other two target wires, by swaps. Second, conjugating an edge Toffoli by X on a wire or by a swap of adjacent wires gives another edge Toffoli. These are sixty derived equations, five for each edge. Third, there are two base relations: $T_1T_4 \approx T_4T_1$, from (T6), and $T_4T_5T_4 \approx T_5T_4T_5$, from (T6) and (T3). Every other Coxeter relation between edge Toffolis onto different wires is one of the two carried by a symmetry of the cube, which is a product of X gates and swaps, through the conjugations of the second group. The generators are $\mathrm{X}_0 \approx T_4T_2T_6T_0$, $\mathrm{CX}_{1\to 0} \approx T_4T_2$, $\mathrm{CCX}_{21\to 0} = T_4$, and $\sigma_0 \approx \mathrm{CX}_{1\to 0}(T_1T_5)\,\mathrm{CX}_{1\to 0}$, where T_1T_5 is the CNOT from wire 0 onto wire 1. The gates one and two wires up follow by conjugation with swaps, and σ_1 by (T3) one wire up, where $\mathrm{CX}_{1\to 2} \approx T_3\,\mathrm{X}_0\,T_3\,\mathrm{X}_0$. So Lemma E.3 applies.

A step of these derivations is a rule of $\mathcal{T}$, a structural rule, a reordering of generators on disjoint wires, or, on three wires, a *two-wire step*. A two-wire step replaces a circuit on two adjacent wires by one with the same function, which is decided by evaluation and justified by completeness on two wires. □

E.4 Placed Gates and Four-Wire Equations

This part follows the modules Placed, ToPlaced, Narrow3, PlacedEngine, and Equations4. On four wires the development works with the 28 *placed gates*: four X_t, twelve $\mathrm{CX}_{c\to t}$, and twelve $\mathrm{CCX}_{ab\to t}$, on distinct wires. The swaps σ_0, σ_1, σ_2 satisfy the Coxeter relations of $\mathrm{Sym}(4)$ by the structural rules, so they form a Coxeter system in which the four wires take the place of the states, and a Coxeter word is a network of swaps. The circuit of a placed gate is the frame $\mathrm{Fr}(b, p)$ of its gate b on the bottom wires by a network p that carries the target and the controls to the wires 0, 1, and 2.

Lemma E.6 (Placed, ToPlaced). *The placed gates have the following properties.*

(1) *The circuit of a placed gate does not depend, modulo $\mathcal{T}$, on the choice of its network, and for a Toffoli gate not on the order of the controls.*
(2) *A gate shifted up by any number of wires is equal modulo $\mathcal{T}$ to a placed gate, and a swap to three placed CNOTs. Hence every four-wire circuit is equal modulo $\mathcal{T}$ to a product of placed gates* (pg-ok).
(3) *Let π be a permutation of the wires, and s a network that carries each wire $\pi(x)$ to wire x. For a placed gate g, the gate πg on the permuted wires is equal modulo $\mathcal{T}$ to $\mathrm{Fr}(g, s)$. So an equation between products of placed gates holds on the wires in any order, and* (T7) *holds for the placed gates on any four wires in any order* (rule7).
(4) *Placed gates on disjoint sets of wires commute modulo $\mathcal{T}$* (disj-comm).
(5) *The circuit of a placed gate denotes the function of the gate* (sem-ok).

Proof. For (1), a gate on the bottom wires commutes with the swaps above it by the structural rules, so Lemma E.4 applies. For the controls, (T5) says that CCX is fixed by the swap of its controls, and the argument of Lemma E.4 applies with σ_1 among the swaps that commute with the gate. For (2), shifting a gate up is conjugating it by the network of a cycle of the wires, by the naturality of the swap, and (T3) writes a swap as three CNOTs. For (3), conjugating a frame by a network gives the frame by the composite network. For (4), one network places both gates, one on the bottom wires and the other shifted above it, and the structural rules exchange them. Item (5) is decided by evaluation for the 28 gates. □

Lemma E.7 (three-wire steps, Narrow3.narrow). *Two products of placed gates that avoid a common wire and denote the same function are equal modulo $\mathcal{T}$. Here a gate avoids a wire that is neither its target nor one of its controls.*

Proof. Relabel the wires by the transposition of the avoided wire and wire 3, with Lemma E.6. The two products are then three-wire circuits with an idle wire on top. They are equal by Proposition E.5, and the structural rules carry the derivation to four wires. □

A four-wire derivation is a list of steps on lists of placed gates. A step rewrites by (T7), on some four wires in some order, or is a *three-wire step*, which replaces consecutive gates that avoid a common wire by gates on the same three wires with the same function. The last condition is decided by evaluation. Between steps, gates on disjoint wires may be reordered. In this way a search derives twelve equations, each in at most 79 steps:

(1) two Toffoli gates with the same controls commute, and so do two with the same target and one common control;
(2) a Toffoli gate commutes with the CNOT onto its target from the fourth wire, in two placements, and with a CNOT from one of its controls onto the fourth wire;
(3) five pairs of Toffoli gates that act on disjoint sets of states commute;
(4) the commutator of $\mathrm{CCX}_{32\to1}$ and $\mathrm{CCX}_{31\to0}$ is $\mathrm{CCX}_{32\to0}$;
(5) $\mathrm{CX}_{1\to2}\,\mathrm{CCX}_{32\to0}\,\mathrm{CX}_{1\to2} \approx \mathrm{CCX}_{32\to0}\,\mathrm{CCX}_{31\to0}$.

The later parts use three of them: the commutation with the CNOT from the fourth wire, the commutation of Toffoli gates with the same target and one common control, and the last equation.

E.5 A Presentation of the Alternating Group

This part follows the modules EvenRS and EvenWord, which work for every N. The alternating group $\mathrm{Alt}(N)$ has index two in $\mathrm{Sym}(N)$, with the coset representatives 1 and s_0. The Reidemeister–Schreier method [48] rewrites the Coxeter presentation of $\mathrm{Sym}(N)$ along these cosets. A letter s_d read in a coset moves to the other coset and leaves a generator behind, which is $s_d s_0$ in the coset of 1, $s_0 s_d$ in the coset of s_0, and trivial for $d = 0$. Writing $c = 0$ for the coset of 1 and $c = 1$ for the coset of s_0, the generators, called *symbols*, are therefore

$$\gamma(0,d) = s_d s_0 \qquad \text{and} \qquad \gamma(1,d) = s_0 s_d \qquad \text{for } 1 \le d \le N-2.$$

The Coxeter word of a word of symbols replaces each symbol by its two letters. The relations are the Coxeter relations rewritten from either coset, without those that become trivial. Writing $\bar{c}$ for the coset other than c, they are

$$\gamma(c,d)\,\gamma(\bar{c},d) = 1, \qquad \gamma(\bar{c},1) = \gamma(c,1)\,\gamma(c,1),$$
$$\gamma(c,d)\,\gamma(\bar{c},d{+}1)\,\gamma(c,d) = \gamma(c,d{+}1)\,\gamma(\bar{c},d)\,\gamma(c,d{+}1), \qquad \gamma(c,e) = \gamma(\bar{c},e) \quad (e \ge 2),$$
$$\gamma(c,d)\,\gamma(\bar{c},e) = \gamma(c,e)\,\gamma(\bar{c},d) \quad (e \ge d+2),$$

for both cosets c and all indices between 1 and $N-2$. Counting each relation once from each coset, there are 28 symbols and 238 relations for $N = 16$, and 12 and 54 for $N = 8$.

Proposition E.8 (rs-complete, rs-sound-sem, rs-≈). *Two words of symbols whose Coxeter words have the same permutation are equal modulo these relations. Conversely, the two sides of each relation have the same permutation, and every Coxeter word with an even number of letters has the permutation of a word of symbols.*

Proof. The Reidemeister–Schreier theorem of the framework (Section 3.2) reduces the first claim to two obligations. Rewriting a Coxeter relation from either coset must give words of symbols

that are equal modulo the relations and end in the same coset, and rewriting the Coxeter word of a symbol from the coset of 1 must give the symbol back. Both are proved for every N, and completeness then transfers from the Coxeter presentation. For the converse, the two sides of each relation have Coxeter words that are equal modulo the Coxeter relations. Rewriting a Coxeter word from the coset of 1 ends in the coset given by the parity of its length (EvenWord), and when it ends in the coset of 1 the Coxeter word of the symbols is equal to the given word modulo the Coxeter relations. □

E.6 Reduction to the Circuits That Keep the Top Wire

This part follows the modules Faithful, ByEvenRS, and Four. A placed gate *keeps* wire m when its target is another wire, so wire m may be a control. Let K_m be the set of products of placed gates that keep wire m, and $K = K_3$. A circuit in K_m preserves the bit of wire m. We call K_m *faithful* if any two of its circuits with the same function are equal modulo $\mathcal{T}$.

Lemma E.9 (Faithful.faithful). *If K is faithful, then K_m is faithful for every wire m.*

Proof. Relabel the wires by the transposition of m and 3, with Lemma E.6. It takes a gate that keeps wire m to one that keeps wire 3, which is decided for all gates and wires. □

Proposition E.10 (Four.complete$_4$). *If K is faithful, then $\mathcal{T}$ is complete on four wires.*

Proof. Number the sixteen states along the reflected Gray code, the eight states of H_0 first. The states numbered d and $d + 1$ then differ on one wire, $t(d)$, which is 0, 1, 0, 2, 0, 1, 0, 3, 0, 1, 0, 2, 0, 1, 0 for $d = 0, \ldots, 14$, and s_d exchanges these two states.

Symbols. For each symbol $\gamma(c, d)$ of Alt(16), a breadth-first search finds a product $R(\gamma)$ of at most eight placed gates that target only the wires 0 and $t(d)$ and that denotes the permutation of the symbol on the numbered states. This is checked state by state. So $R(\gamma)$ keeps every other wire.

Relations. A relation involves symbols with at most two values d and e of the index. The gates of its two sides target at most the wires 0, $t(d)$, and $t(e)$, so both sides lie in K_m for some m among 1, 2, and 3. They denote the same function by Proposition E.8, and they are equal modulo $\mathcal{T}$ because K_m is faithful, by Lemma E.9.

Gates. Every placed gate is equal modulo $\mathcal{T}$ to the product of the $R(\gamma)$ along a word of symbols. For 24 of the gates there is a wire m among 1, 2, and 3 that the gate keeps and such that the gate permutes the numbers inside each run of consecutive numbers on which the bit of wire m is constant. A Coxeter word for the gate inside these runs uses only the s_d with $t(d) \neq m$, its symbols keep wire m, and the faithfulness of K_m gives the equation. The four other gates are X_3, $\mathrm{CX}_{0\to3}$, $\mathrm{CX}_{1\to3}$, and $\mathrm{CCX}_{10\to3}$. Each is equal, by a three-wire step, to a product of gates translated before.

Conclusion. Let u and w be products of placed gates with the same function. Their words of symbols have the same permutation, so they are equal modulo the relations by Proposition E.8. The relations hold for the products $R(\gamma)$, so u and w are equal modulo $\mathcal{T}$. Every four-wire circuit is a product of placed gates by Lemma E.6. □

E.7 Controlled Matchings

This part follows the modules PFrames, CBase, CFrame, CLabels, and CSem. From here on, a *position* is the number of a state of the lower wires along the Gray code of Appendix E.3, and each half is a copy of the eight positions. The edge Toffolis $T_0, \ldots, T_6$ of the lower wires, with wire 3 idle, form a Coxeter system inside four wires, so Lemmas E.3 and E.4 apply to the circuits $\langle p \rangle$ of the Coxeter words p of Sym(8), and every circuit on the lower wires is equal modulo $\mathcal{T}$ to such a circuit.

A *perfect matching* partitions the eight positions into four pairs, and we identify it with the permutation that exchanges the two positions of each pair. There are 105 perfect matchings. The

gate $\mathrm{CX}_{3\to0}$ is the identity on H_0 and acts on H_1 as the perfect matching $\mu_0 = (0\ 1)(2\ 3)(4\ 5)(6\ 7)$, which flips wire 0. Its frame $\mathrm{Fr}(\mathrm{CX}_{3\to0}, p)$ is the identity on H_0 and acts on H_1 as the perfect matching $p\,\mu_0\,p^{-1}$.

Lemma E.11 (CFrame.W-core, CFrame.W′). *If the permutation of a Coxeter word r commutes with μ_0, then $\langle r\rangle$ commutes with $\mathrm{CX}_{3\to0}$ modulo $\mathcal{T}$. Hence two frames of $\mathrm{CX}_{3\to0}$ with the same perfect matching are equal modulo $\mathcal{T}$.*

Proof. A position is one of the two sides of a pair of μ_0, and the four pairs are the four values of the wires 2 and 1. A permutation that commutes with μ_0 permutes the four pairs and exchanges the sides in some of them. So r has the permutation of a product of the edge Toffolis T_0, T_2, T_4, and T_6 onto wire 0, one for each pair whose sides are exchanged, and of one of 24 fixed Coxeter words that permute the pairs. By Lemma E.3, $\langle r\rangle$ is equal to the circuit of that product. The gate $T_4 = \mathrm{CCX}_{21\to0}$ commutes with $\mathrm{CX}_{3\to0}$ by the four-wire equation for the CNOT onto the target from the fourth wire, and the other three edge Toffolis are T_4 between X gates on its controls, which commute with $\mathrm{CX}_{3\to0}$. The circuit of each of the 24 words has the function of a circuit on the wires 1 and 2. It is equal to that circuit by a three-wire step, and that circuit commutes with $\mathrm{CX}_{3\to0}$ because they share no wire. For the second claim, if $\mathrm{Fr}(\mathrm{CX}_{3\to0}, p)$ and $\mathrm{Fr}(\mathrm{CX}_{3\to0}, p')$ have the same matching, then the permutation of $p'^{-1}p$ commutes with μ_0, and the frames are equal as in the proof of Lemma E.4. The two matchings are compared on the eight positions by evaluation. □

For each perfect matching μ, the development fixes a Coxeter word p_μ with $p_\mu\,\mu_0\,p_\mu^{-1} = \mu$, and the *controlled matching* of μ is $M_\mu = \mathrm{Fr}(\mathrm{CX}_{3\to0}, p_\mu)$. By Lemma E.11 it does not depend on the choice of p_μ, so $\mathrm{CX}_{3\to0} \approx M_{\mu_0}$. It is the identity on H_0 and acts as μ on H_1, which is decided for the 105 matchings at once. The union of a matching μ with μ_0 is a set of cycles, and the numbers of pairs of μ_0 on these cycles form a partition of 4, the *type* of μ. One matching is distinguished for each type: μ_0 for 1 + 1 + 1 + 1, and one each for 2 + 2, 2 + 1 + 1, 3 + 1, and 4. A table gives every matching as the distinguished matching of its type conjugated by a Coxeter word whose permutation commutes with μ_0.

E.8 Relations between Controlled Matchings

This part follows the modules CRel, CBC, and BaseComm.

Lemma E.12 (X-inv, X-conj, X-klein). *For perfect matchings μ and ν:*

(1) $M_\mu M_\mu \approx \varepsilon$;
(2) $M_\mu M_\nu M_\mu \approx M_{\mu\nu\mu}$;
(3) $M_\mu M_\nu \approx M_{\mu\nu}$ *if* $\mu \neq \nu$ *commute and* $\mu\nu$ *is a perfect matching.*

In the development, (2) *and* (3) *each take a hypothesis, a Boolean computed from the two matchings. It says that the frames and the table entries that the proof below uses have the expected matchings. Agda evaluates it for each pair to which a later proof applies the relation, and the development does not prove that it holds for all pairs.*

Proof. Item (1) holds because $\mathrm{CX}_{3\to0}$ is an involution, by a three-wire step.

For item (2), let $\overline{\mathrm{CX}}_{3\to0} = \mathrm{X}_3\,\mathrm{CX}_{3\to0}\,\mathrm{X}_3$, which flips wire 0 when wire 3 is 0. Then $\mathrm{CX}_{3\to0} \approx \mathrm{X}_0\,\overline{\mathrm{CX}}_{3\to0} \approx \overline{\mathrm{CX}}_{3\to0}\,\mathrm{X}_0$ by a three-wire step, so $M_\mu \approx P_\mu \bar{M}_\mu \approx \bar{M}_\mu P_\mu$, where $P_\mu = \mathrm{Fr}(\mathrm{X}_0, p_\mu)$ applies μ in both halves and $\bar{M}_\mu = \mathrm{Fr}(\overline{\mathrm{CX}}_{3\to0}, p_\mu)$ applies μ in H_0 only. Since $\mathrm{X}_0 \approx T_0T_2T_4T_6$, the circuit P_μ is the circuit of a Coxeter word, and also of the inverse word.

The circuits $\bar{M}_\mu$ and M_ν act in different halves, and we derive that they commute. Conjugating by $\langle p_\mu\rangle$ turns the two into $\overline{\mathrm{CX}}_{3\to0}$ and a frame of $\mathrm{CX}_{3\to0}$. By the table of Appendix E.7 and Lemma E.11,

that frame is the frame of a distinguished matching conjugated by a circuit $\langle c \rangle$ whose permutation commutes with μ_0. Such a circuit commutes with $\mathrm{CX}_{3\to 0}$, by Lemma E.11, and with X_3, hence with $\overline{\mathrm{CX}}_{3\to 0}$. So it suffices that $\overline{\mathrm{CX}}_{3\to 0}$ commutes with the frame of each distinguished matching. These are the five *base commutations*. For the types $1+1+1+1$ and $2+2$ they are three-wire steps. For the other three types they are four-wire derivations of 171, 496, and 187 steps, which a search finds.

Given the commutation, $M_\mu M_\nu M_\mu \approx P_\mu \bar{M}_\mu M_\nu \bar{M}_\mu P_\mu \approx P_\mu M_\nu P_\mu$. This is a frame of $\mathrm{CX}_{3\to 0}$ with the matching $\mu\nu\mu$, and Lemma E.11 concludes.

For item (3), the matchings μ, ν, and $\mu\nu$ form, with the identity, a group of four permutations that acts on the eight positions with two orbits of four positions. The frame word of the list of the two orbits carries a standard pair of matchings to μ and ν. The standard pair is μ_0 and its conjugate by $\mathrm{CX}_{0\to 1}$, and their product flips wire 1. For the standard pair the relation is a three-wire step on the wires 0, 1, and 3. □

E.9 The Controlled Alternating Group

This part follows the modules LabelEngine, CCerts, and CFaithful.

Proposition E.13 (CFaithful.cfaith). *Two products of controlled matchings with the same function are equal modulo $\mathcal{T}$.*

Proof. The controlled matchings act on H_1 as even permutations of the eight positions, and the argument is that of Proposition E.10 with the presentation of Alt(8) from Appendix E.5.

Symbols. Each symbol γ is given a word $\Phi(\gamma)$ of two or four matchings whose product is the permutation of γ, which is decided on the eight positions. For $d \geq 2$, the symbol $\gamma(c, d)$ is a double transposition $(i\ j)(k\ l)$, the product of the commuting matchings $(i\ k)(j\ l)\rho$ and $(i\ l)(j\ k)\rho$, where ρ is a fixed matching of the other four positions. For $d = 1$, the symbol is a three-cycle, the product of two double transpositions that share the pair $(6\ 7)$.

Certificates. A word of matchings is rewritten by moves. A conjugation move rewrites $\mu\nu$ to $(\mu\nu\mu)\mu$ or to $\nu(\nu\mu\nu)$. A cancellation move deletes $\mu\mu$, and an insertion move inserts it. A Klein move merges $\mu\nu$ into the matching $\mu\nu$, or splits it. Each move is an instance of Lemma E.12 for the controlled matchings, once its side condition evaluates to true. A *certificate* is a list of moves, and Agda checks it by running it.

Relations. For each of the 54 relations of Alt(8), a search finds a certificate between the images under Φ of its two sides. These 54 certificates have 412 moves in all. Twelve further certificates show that the word of a symbol reversed is the word of the inverse symbol.

Matchings. Every controlled matching is equal modulo $\mathcal{T}$ to the image under Φ of a word of symbols. For μ_0, the word is $\gamma(0,2)\gamma(0,4)\gamma(0,6)$, by a certificate of eight moves. Every other matching is the conjugate of its parent, in a breadth-first tree of depth nine, by the word of a symbol, and the conjugation is Lemma E.12(2), applied matching by matching.

Conclusion. Two products of controlled matchings with the same function translate into words of symbols with the same permutation. These are equal modulo the relations by Proposition E.8, and the certificates carry the equality back. □

E.10 Faithfulness of K and Completeness on Four Wires

This part follows the modules KData, KFaithful, and Completeness.

Proposition E.14 (KFaithful.faithful$_3$). *The set K is faithful.*

Proof. *Gates.* Every gate of K is equal modulo $\mathcal{T}$ to a product of gates on the lower wires and $\mathrm{CX}_{3\to 0}$. The gates $\mathrm{CX}_{3\to 1}$ and $\mathrm{CX}_{3\to 2}$ are $\mathrm{CX}_{3\to 0}$ between two swaps of lower wires. Let a and t be two lower wires and b the third. The Toffoli gate with controls a and 3 and target t is equal to

$\mathrm{CX}_{3\to b}\,\mathrm{CCX}_{ab\to t}\,\mathrm{CX}_{3\to b}\,\mathrm{CCX}_{ab\to t}$. This is the last equation of Appendix E.4 with the commutation of two Toffoli gates that have the same target and one common control, both relabeled.

Normal shape. A gate g on the lower wires passes a controlled matching: $g\,M_\mu \approx M_{\mu'}\,g$, where μ' is the conjugate of μ by g. Indeed g is the circuit of a Coxeter word, by a three-wire step, and Lemma E.11 identifies the conjugated frame. This is decided for the twelve gates on the lower wires and the 105 matchings. Since $\mathrm{CX}_{3\to 0} \approx M_{\mu_0}$, every circuit in K is equal modulo $\mathcal{T}$ to a product of controlled matchings followed by a product of gates on the lower wires.

Faithfulness. Let two such circuits have the same function. Controlled matchings are the identity on H_0, so the two products on the lower wires agree on H_0. They do not see wire 3, so they agree everywhere, and they are equal by a three-wire step (Lemma E.7). Every placed gate is an involution, again by a three-wire step, so the two products on the lower wires cancel. The two products of controlled matchings then have the same function, and they are equal by Proposition E.13. □

Propositions E.10 and E.14 give completeness on four wires (complete). With Propositions E.1 and E.5, this proves the first two claims of Theorem 6.1. Equation (T7) enters only through the four-wire equations: in Lemma E.11, in the base commutations of Lemma E.12, and in the gates of K.

E.11 The Groups of Circuits

This part follows the modules ByPerm and Groups.

Proposition E.15 (presentation$_1$–presentation$_4$). *For $1 \le k \le 4$, G_k is a group. The map $u \mapsto [\![u]\!]$ is an isomorphism from G_k onto $\mathrm{Sym}(2^k)$ for $1 \le k \le 3$, and from G_4 onto $\mathrm{Alt}(16)$.*

Proof. Number the states as in the completeness proof, and read a circuit as the permutation that it carries out on the numbers. This is a homomorphism by the semantics. It is well defined on G_k by Proposition E.1 and injective by completeness. Every generator applied twice has the function of ε, so by completeness it is its own inverse, and G_k is a group. For $k \le 3$ the map is onto because every permutation is that of a Coxeter word, whose circuit carries it out. On four wires, every circuit is equal to the product of the $R(\gamma)$ along a word of symbols, and each symbol is a product of two transpositions, so every circuit carries out an even permutation. Conversely, an even permutation is that of a Coxeter word with an even number of letters, which rewrites into symbols by Proposition E.8, and the product of their $R(\gamma)$ carries it out. □

E.12 The Block Normal Form

This part follows the modules NormalForm, NFLib, NFTables, and EvenWord. A state of four wires is given by its half and by the position of its lower wires, and the sixteen states are numbered as in Appendix E.6, so that the numbers below 8 are the states of H_0.

The development fixes the three circuits as follows. $\mathrm{D}(B)$ is the product of the edge Toffolis of the lower wires along the Coxeter word that the library gives for B. For $\mathrm{C}(E)$, the Coxeter word that the library gives for E is rewritten into symbols of $\mathrm{Alt}(8)$ by Proposition E.8, and $\mathrm{C}(E)$ is the product of the controlled matchings of their words $\Phi(\gamma)$. So $\mathrm{C}(E)$ lies in K. For r_S, list the states of S in increasing order of their numbers, followed by the other states in increasing order, and take the frame word of this list, which sends each state to the state numbered by its place in the list. If this word has an odd number of letters, it is followed by s_{14}, which exchanges the last two states. The word is rewritten into symbols of $\mathrm{Alt}(16)$, and r_S is the product of their $R(\gamma)$. In Agda the three circuits are opaque, and their definitions are available as equations.

Lemma E.16 (d-sem, c-low, c-high, r-sem, nf-sem). *The circuit $\mathrm{D}(B)$ preserves the halves, with the blocks B and B. The circuit $\mathrm{C}(E)$ preserves the halves, with the blocks the identity and E. The circuit*

r_S maps a state into H_0 exactly when the state is in S. Hence $\mathrm{NF}(E,B,S)$ *is r_S, which maps S onto H_0, followed by a circuit with the blocks B and BE.*

Proof. The circuit of a Coxeter word carries out its permutation on the lower wires and does not see wire 3. The controlled matchings are the identity on H_0 and act on H_1 as their matchings, whose product along Φ is the permutation of the symbols. The list of S has the eight states of S in its first eight places, and s_{14} preserves the halves. □

Proposition E.17 (unique, onto). *Two normal forms with the same function have the same coordinates, and every even permutation of the sixteen states is the function of a normal form. So* $(E,B,S) \mapsto [\![\mathrm{NF}(E,B,S)]\!]$ *is a bijection from* $\mathrm{Alt}(8) \times \mathrm{Sym}(8) \times \binom{\mathbb{F}_2^4}{8}$ *onto* $\mathrm{Alt}(16)$.

Proof. By Lemma E.16, a normal form maps a state into H_0 exactly when the state is in S, so its function determines S and hence r_S. Evaluating at the state that r_S maps to a given position of H_0 or of H_1 then gives B and BE, hence E. The second claim follows from the first claim of Proposition E.18 below, applied to a circuit that carries out the permutation, which exists by Proposition E.15. The development also checks the count $12870 \cdot 8! \cdot 8!/2 = 16!/2$. □

Proposition E.18 (normal-form, NormalForm.unique). *Let u be a four-wire circuit, S the set of states that u maps into H_0, B_0 and B_1 the blocks of $r_S^{-1}u$, and $E = B_0^{-1}B_1$. Then S has eight states, E is even, and $u \approx \mathrm{NF}(E,B_0,S)$. If $u \approx \mathrm{NF}(E',B',S')$, then $(E',B',S') = (E,B_0,S)$.*

Proof. The function of u is a permutation, so S has as many states as H_0. The permutation $r_S^{-1}u$ maps H_0 onto S and then back onto H_0, so it preserves the halves. The circuit that applies $\mathrm{D}(B_0^{-1})$, then r_S reversed, and then u is the identity on H_0 and acts as E on H_1. It carries out an even permutation of the sixteen numbers by Proposition E.15, and a permutation that fixes the numbers below 8 and acts as E on the others has the sign of E. So E is even. By Lemma E.16, $\mathrm{NF}(E,B_0,S)$ has the function of u, and completeness gives $u \approx \mathrm{NF}(E,B_0,S)$. The last claim follows from Proposition E.1 and the first claim of Proposition E.17. □

Proposition E.19 (absorption, NormalForm.Absorb). *Let w be a four-wire circuit and* $\mathrm{NF}(E,B,S)$ *a normal form. Let S' be the set of states that w maps into S, and A_0 and A_1 the blocks of $r_{S'}^{-1}\, w\, r_S$. Then $w\,\mathrm{NF}(E,B,S) \approx \mathrm{NF}(E',B',S')$ with $B' = A_0B$ and $B'E' = A_1BE$.*

Proof. The coordinates (E',B',S') are those that Proposition E.18 gives for the circuit $w\,\mathrm{NF}(E,B,S)$, and the equation is that proposition. It remains to compute them. A state is mapped into H_0 by $w\,\mathrm{NF}(E,B,S)$ exactly when w maps it into S. The permutation $r_{S'}^{-1}\, w\, r_S$ maps H_0 onto S', then onto S, and then onto H_0, so it preserves the halves. Hence $r_{S'}^{-1}\, w\,\mathrm{NF}(E,B,S)$ has the blocks A_0B and A_1BE. The development states the two formulas pointwise, as $B'(x) = B(A_0(x))$ and $E'(B'(x)) = E(B(A_1(x)))$ for the positions x. □

Proposition E.15 is the last claim of Theorem 6.1, and Propositions E.17 to E.19 are the three items of Theorem 6.2.

E.13 Implementation

The development consists of 40 modules in `Examples/Groups/XCXCCX-4bit`, all reached from MainTheorem, and every module is checked with Agda's `--safe` flag. Given the interfaces of the shared library, Agda checks them in 85 seconds, the sum of the times it reports for the modules (98 seconds of wall time, with a peak resident size of 1.9 GB). Scripts in the development generate the long derivations, namely those on at most three wires, the four-wire equations, and the three base commutations of Lemma E.12, together with the tables of matchings and the 412 moves of the

certificates. Agda checks each derivation step and each move by evaluation. Two traps took most of the debugging time, and both made Agda evaluate the symbolic word that the library's symmetric group development builds for an arbitrary permutation. The first is comparing two spellings of the sign of a permutation at a numeric width, such as 16 against 8 + 8. The second is meeting a goal about a circuit with a lemma about its unfolding. The development therefore reasons about signs at variable widths and keeps the normal form's building blocks opaque, with their definitions given as equations. Its Lean translation, `QCert/XCXCCX4bit`, makes the same choices, and Lean checks the same derivations and certificates in the kernel (Appendix C).